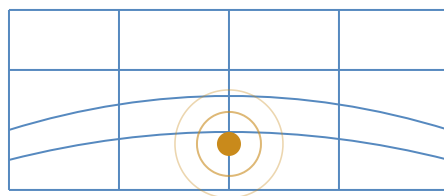


The Mathematics of Constraint

A From-Scratch Primer for Reading the Research Program —

Every Equation, Every Intuition, Assuming Only High-School Algebra



Prepared for the research program on geometric convergence,
weakness-based generalization, and machine agency.

A self-contained companion — read start to finish, or jump to the chapter
behind whatever equation stopped you.

What This Book Is, and How to Use It

Your research program spans roughly fifty papers, and its ideas are carried almost entirely by mathematics — group theory, differential geometry, information theory, causal inference, control theory, statistics. If your last math was Algebra 2 and a statistics course a decade and a half ago, those papers are written in a language you were never taught to read. This book teaches that language, from the ground up, using *your own papers' equations as the worked examples*.

It assumes only what you said you have: algebra, a little trig, faint memories of stats. From there it builds every concept the papers use — no step skipped, nothing "left to the reader." The goal is not to make you a mathematician; it is to make you **fluent enough to open any paper in the repository, decode its central equation, judge whether its claim is earned, and explain it to someone else.**

How the book is organized. Part I teaches you to *read* notation at all — the survival skill everything else depends on. Parts II–V build the five mathematical worlds the program lives in: geometry, change/optimization/information, symmetry/shape, and agency/dynamics/ causality. Part VI assembles those tools into the program's own inventions — weakness, the core theorems — and closes with a full walkthrough of reading one real paper and a glossary you can use as a decoder ring.

How to read the boxes

Throughout, colored boxes do consistent jobs, so you can skim by color: **Definition** pins down a term precisely. **Intuition pump** gives the mental picture that makes it stick. **Visual proof** shows why something is true with a figure, not a derivation. **In our papers** connects the idea straight to an equation from your research. **Pitfall** flags a place the notation misleads or a claim is subtler than it looks. Every chapter ends with a "What you can now read" inventory — a checklist of notation you have just unlocked.

Table of Contents

Part I · How to Read Mathematics at All

- 1 · The Symbol Survival Kit
- 2 · Functions, Spaces, and Many Dimensions
- 3 · Exponents and Logarithms: Doubling
- 4 · Counting: the Size of "All the Ways"
- 5 · Probability: the Grammar of Uncertainty
- 6 · Statistics: Earning the Right to a Claim

Part II · The Geometry of Meaning

- 7 · Vectors, Dot Products, Cosine Similarity
- 8 · Matrices, Spectra, the Shape of a Cloud
- 9 · Metrics, Curvature, and Gauge

Part III · Change, Optimization, Information

- 10 · Derivatives and Gradient Descent
- 11 · Integrals and Constrained Optimization
- 12 · Information Theory: Bits and Surprise
- 13 · Waves and Spectra: Fourier Thinking

Part IV · Symmetry and Shape

- 14 · Groups, Symmetry, and Equivariance
- 15 · Topology: Holes, Tori, and Betti Numbers

Part V · Agency, Dynamics, Causality

- 16 · Dynamical Systems, Control, Attractors
- 17 · Decisions, Value, and Reinforcement Learning
- 18 · Causality, Interventions, Identifiability

Part VI · The Program's Own Mathematics

19 · Weakness, from First Principles

20 · The Core Theorems Assembled

21 · Reading a Real Paper, End to End

22 · Glossary and Symbol Decoder Ring

PART I

How to Read Mathematics at All

Mathematical notation is not knowledge — it is *handwriting*. Every equation in every paper in this research program is a compressed English sentence, written in a shorthand that took mathematicians a few centuries to standardize. The single biggest unlock for reading the papers is not learning more math; it is learning to *decompress the shorthand back into English* without panicking. This part gives you the decompression tools. Everything later in the book builds on it.

CHAPTER 1

The Symbol Survival Kit

Here is a secret that changes how the papers feel: **there are only about forty symbols in active use across the entire research program**, and most of them are doing one of four jobs — naming things, grouping things, comparing things, or combining things. Once you can sight-read those forty, an equation stops being a wall and becomes a sentence with unfamiliar vocabulary, which is a much smaller problem.

1.1 Letters are just variable names

In algebra you met x and y . Research papers use far more letters, but the rule is the same: **a letter is a name for a quantity, chosen by the author, with no inherent meaning**. Papers reuse certain letters by convention, the way programmers name a loop counter `i`. Conventions in our papers:

Letters	Conventional job in our papers
x, y, z	inputs, outputs, data points ("an example the system sees")
h, f, g	functions — especially h for a <i>hypothesis</i> , a candidate rule the learner might adopt
n, m, k, d	whole-number counts and sizes: number of examples, number of dimensions
i, j, t	indices — "which one we're talking about" (t almost always means a time step)
w, v, u	vectors or weights (lists of numbers inside a model); in the weakness papers $w(h)$ is instead the <i>weakness score</i> of hypothesis h — context decides
p, q	probabilities or probability distributions
s, a, r	state, action, reward — the agency and control papers
$L, \mathcal{L}$	a loss — the number training tries to shrink
CAPITALS: X, H, S, D	whole <i>collections</i> (sets): all possible inputs, all hypotheses, all states, a dataset

Greek letters are not fancier math — they are just *more names*, used when the Latin alphabet runs out or when convention has claimed a letter for a specific role. These are the ones our papers actually use:

Symbol	Name	Typical job here
α (alpha), β (beta)	alpha, beta	tuning knobs: learning rates, mixing weights, trade-off strengths
γ	gamma	the discount factor in decision problems (Chapter 16) — how much the future counts
ϵ	epsilon	a deliberately small number: noise size, tolerance, exploration rate
θ	theta	the whole bundle of a model's adjustable parameters ("the knobs")
λ	lambda	a strength dial on a penalty or constraint; also an eigenvalue (Chapter 8)

μ	mu	a mean (average); sometimes a setpoint in control chapters
σ	sigma (lowercase)	standard deviation — the typical size of wobble around the mean; also the squashing function in neural nets
Σ	sigma (capital)	"add these all up" (Section 1.4); as a bare symbol, a covariance matrix
Π	pi (capital)	"multiply these all together"
η	eta	learning rate — step size of gradient descent (Chapter 10)
δ, Δ	delta	a change or difference: Δx = "the change in x "
ρ	rho	a correlation (Chapter 6) or a density
τ	tau	a threshold or a time constant
ϕ, ψ	phi, psi	feature maps and embeddings — functions that turn raw things into vectors
π	pi (lowercase)	3.14159... only in geometry; in the agency papers it is a <i>policy</i> , an agent's rule for choosing actions

Intuition pump — letters are code, not content

When you read code, `customer_count` and `n` can hold the same value; the name is for the reader. Math is identical, but with a display constraint: equations were designed to be written in chalk, so names got squeezed to one character plus decorations. $\hat{y}$ is just the one-character version of `predicted_y`. Reading an equation is reading minified code — and this chapter is the un-minifier.

1.2 Decorations: hats, bars, stars, subscripts, superscripts

Because names must be one letter, authors attach *decorations* to make families of related names. None of these change what the base letter is; they annotate it.

Decoration	Read as	Meaning
x_i	"x sub i"	the i -th item in a list: $x_1, x_2, x_3, \dots$ If a paper says "trial i ", then x_i is that trial's input.
x^2	"x squared"	usually a power, as in algebra — but see the warning below
$x^{(t)}, s_t$	"x at step t"	the value at time step t ; parenthesized superscripts are version labels, <i>not</i> powers
$\hat{x}$ ("x-hat")	"x hat"	an <i>estimate or prediction</i> of x : $\hat{y}$ is the model's guess at the true y
$\bar{x}$ ("x-bar")	"x bar"	the <i>average</i> of a bunch of x 's
x' ("x-prime")	"x prime"	"another one of the same kind": if s is the current state, s' is the next state. (In calculus f' is the derivative — context decides.)
x^*	"x star"	the <i>optimal</i> or special one: h^* is the best hypothesis, the one that actually generated the data
$\tilde{x}$ ("x-tilde")	"x tilde"	a perturbed, approximated, or transformed cousin of x

x (bold)	—	a vector (a list of numbers) rather than a single number; many of our papers don't bother with bold and let context decide
-----------------	---	--

Pitfall — superscripts are overloaded

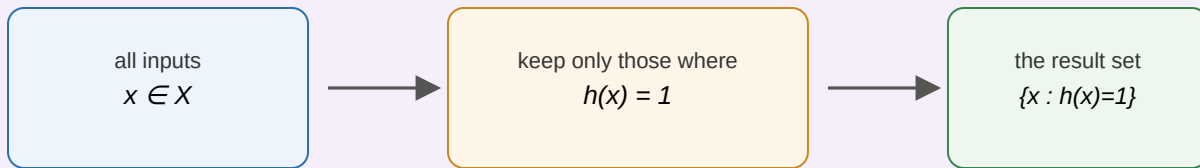
In σ^2 (variance) the 2 is a genuine power. In $\theta^{(t)}$ the (t) is a time-step label. In $\mathbb{R}^d$ the d counts dimensions ("lists of d real numbers"). Rule of thumb: a parenthesized superscript is a label; a superscript on a set symbol like $\mathbb{R}$ is a dimension count; otherwise it is probably a power.

1.3 Sets: the language of "all the things"

A **set** is just an unordered collection with no duplicates — a club with a membership list. Set notation appears constantly in the papers because science is about *classes* of situations, not single cases.

Notation	Read aloud as	Plain meaning
$\{1, 3, 5\}$	"the set containing 1, 3, 5"	curly braces list members
$x \in A$	"x is in A" / "x is an element of A"	membership. $x \notin A$: not a member
$A \subseteq B$	"A is a subset of B"	every member of A is also in B
$A \cup B, A \cap B$	"A union B", "A intersect B"	everything in either; only what's in both
$ A $	"the size of A"	how many members A has — this exact notation is the heart of the weakness papers
$\emptyset$	"the empty set"	the club with no members
$\{x \in X : P(x)\}$	"the set of x in X such that P holds"	set-builder notation — a filter. Read the colon (or a vertical bar) as "such that"
$\mathbb{R}, \mathbb{R}^d$	"the reals", "R-d"	all decimal numbers; all lists of d decimal numbers (= d -dimensional space, Chapter 7)
$\mathbb{N}, \mathbb{Z}$	"the naturals", "the integers"	counting numbers 0,1,2,...; all whole numbers including negatives
$[0, 1]$	"the interval zero-one"	all decimals from 0 to 1 inclusive — where probabilities live
$X \times Y$	"X cross Y"	all <i>pairs</i> (x, y) — e.g. State $\times$ Action = every state-action combination

Visual proof — set-builder notation is a filter pipeline



Reading $\{x \in X : h(x) = 1\}$. Start with every possible input, run each through the test after the colon, keep the ones that pass. The result is a set, and $|\cdot|$ then counts it. This one expression, plus a logarithm, is the core definition of the entire weakness research arc.

1.4 Big operators: Σ , Π , and friends

Σ (capital sigma) means "add up all of these"; it is a `for`-loop that accumulates a sum. The decorations below and above say where the loop starts and stops; the expression to the right is the loop body.

$$\sum_{i=1}^n x_i = x_1 + x_2 + \cdots + x_n$$

"Loop i from 1 to n , adding up the x_i 's." In code: `total = 0; for i in 1..n: total += x[i]`

Π (capital pi) is the same loop but multiplying instead of adding. Two more "loop-like" operators do *selection* instead of accumulation:

Notation	Read as	Plain meaning
$\max_a f(a)$	"the max over a of $f(a)$ "	try every allowed a , report the <i>largest value</i> f reaches
$\operatorname{argmax}_a f(a)$	"the arg-max over a"	try every a , report <i>which</i> a achieved that largest value — the winner's name, not its score. "arg" = argument = input
$\min, \operatorname{argmin}$	—	same, for smallest. Training a model is one giant argmin: find the knob settings that make the loss smallest
$\mathbb{E}[X], \mathbb{E}_{x \sim p}[f(x)]$	"the expected value of X "	the long-run average (Chapter 5): what $f(x)$ averages out to when x is drawn from distribution p
$\forall x, \exists x$	"for all x ", "there exists an x "	logic quantifiers: "this holds for every x " / "at least one x makes this hold"

1.5 Relations: the verbs of math

Symbol	Read as	Nuance
=	"equals"	the two sides are the same number/object
$:=$ or $\triangleq$ or $\equiv$	"is defined as"	not a discovered fact — the author is <i>coining a name</i> . When you see $:=$, relax: nothing is being claimed, only labeled
$\approx$	"is approximately"	close enough for the argument being made
$\propto$	"is proportional to"	equal up to a constant multiplier we don't care about right now
$\geq, \leq, \gg, \ll$	"at least / at most / much bigger / much smaller"	the doubled versions are qualitative claims of scale
$\rightarrow$	"maps to" / "goes to"	in $f: X \rightarrow Y$, f turns members of X into members of Y ; in limits, "approaches"
$x \sim p$	"x is drawn from p"	x is a random sample from distribution p (Chapter 5)
$\Rightarrow, \Leftrightarrow$	"implies", "if and only if"	one-way and two-way logical consequence

1.6 The five-step method for reading any equation

Here is the procedure this whole book trains. It works on every equation in every paper.

- Find the verb.** Locate the $=$, $\leq$, $:=$, or $\propto$. Everything is now "left thing *verb* right thing." If the verb is $:=$, the equation is a definition — a naming ceremony, not a claim.
- Name the nouns.** Identify each letter. The paper *always* defines them within a few lines of first use — hunt for the phrase "where ... denotes ...". Keep a scrap-paper glossary as you read.
- Read the loops.** Expand each Σ , Π , $\max$, $\mathbb{E}$ mentally into "for each ..., accumulate/select ...".
- Say it in English.** Force a full sentence out loud. Clumsy is fine; incomplete is not.
- Test it on a toy case.** Plug in the smallest example you can invent ($n = 2$, a coin flip, a 2-pixel image). If the sentence survives the toy case, you own the equation.

In our papers — the method applied to the program's most important definition

The weakness arc (Chapters 18 and 20) rests on one definition, from the flagship weakness-invariance paper:

$$w(h) := \log_2 |\{x \in X : h(x) = 1\}|$$

Step 1 (verb): " $:=$ " — this is a definition; nobody is claiming anything yet. **Step 2 (nouns):** h is a candidate rule (hypothesis); X is the set of all possible inputs; $h(x) = 1$ means "rule h says yes to input x ." **Step 3 (loops):** the set-builder is a filter: collect every input the rule accepts. $|\cdot|$ counts them. $\log_2$ converts the count to bits (Chapter 3). **Step 4 (English):** "*The weakness of a rule is the number of doublings in how many inputs it says yes to.*" A rule that accepts many inputs is *weak* (it barely constrains the world); a rule that accepts few is *strong*. **Step 5 (toy case):** with 8 possible inputs, a rule accepting all 8 has weakness $\log_2 8 = 3$; a rule accepting exactly one has weakness $\log_2 1 = 0$.

The papers' headline claim — *weak-but-compatible rules generalize better than merely short ones* — is a claim about this number, and you just read it.

Intuition pump — equations are frozen sentences

Nobody thinks in notation, including the people who write it. The author had an English thought ("count how many inputs the rule tolerates, then take the log"), froze it into symbols for precision and space, and shipped it. Your job as a reader is thawing, not decoding secret wisdom. If a thawed sentence sounds almost dumb — "add up the errors and divide by how many there are" — you have probably read it *correctly*. Most working equations are that plain.

1.7 What you can now read

You can sight-read: subscripts as list indices, hats as estimates, stars as optima, set-builder filters, $|\cdot|$ as counting, Σ as a summing loop, argmax as "the winner", $\mathbb{E}$ as long-run average, $:=$ as a naming ceremony, and $x \sim p$ as sampling. That is roughly 80% of all notation in the repository's papers by volume. The rest of this book supplies the concepts behind the remaining 20%.

CHAPTER 2

Functions, Spaces, and Living in Many Dimensions

You met functions in Algebra 2 as things like $f(x) = 2x + 3$: plug in a number, get a number. The papers use functions constantly, but almost never as formulas. This chapter upgrades your picture of a function from "a formula" to "any reliable input $\rightarrow$ output machine," and then upgrades your picture of space from "the xy-plane" to "any place where a thing can be described by a list of numbers." Those two upgrades unlock most of the program's vocabulary.

2.1 A function is a machine with a contract

The modern definition: a **function** is *any* rule that assigns to each input exactly one output. No formula required. A lookup table is a function. A trained neural network is a function. "The reward the environment gives when an agent eats item x while at energy E " is a function. The notation

$$f: X \rightarrow Y$$

is the function's *type signature*, exactly like a function signature in code: "f takes things from set X and returns things from set Y." It tells you the contract before you know anything about the internals. Papers state signatures constantly, and they are gifts to the reader — pure orientation, zero difficulty. Examples straight from our papers:

Signature in the paper	What it tells you
$h: X \rightarrow \{0, 1\}$	a hypothesis takes an input and answers yes (1) or no (0) — a classifier
$\kappa: U \rightarrow \mathbb{R}_{\geq 0}$	"concern" assigns a nonnegative number (how much this outcome matters) to each deployment situation
$self_head: (z, E, a) \rightarrow \Delta E$	a small network takes a latent vector, an energy level, and an action, and predicts a change in energy
$G_\phi: (x, z) \rightarrow \theta$	a generator takes an image and a random seed and outputs a rotation angle

Intuition pump — functions as vending machines

A function is a vending machine with a guarantee: same button, same snack, every time. It does not matter whether the machine works by formula, by lookup table, or by a trillion-parameter neural network; from the outside, all that exists is the button $\rightarrow$ snack contract. When a paper says "we train a function f_θ ," it means: we built a vending machine whose wiring θ we can adjust, and we adjusted the wiring until the snacks came out right on the training examples. The scientific question is always what the machine does on *buttons it was never tested on* — that is generalization, and it is the central obsession of this research program.

2.2 Composition: machines feeding machines

Writing $g(f(x))$ — or $(g \circ f)(x)$, "g after f" — means run x through f , then feed the result to g . Every deep learning system in the papers is a composition: an *encoder* turns raw input into a vector $z = enc(x)$, then a *head* turns that vector into a prediction $\hat{y} = head(z)$. When a paper freezes the encoder and retrains the head, or ablates the encoder's

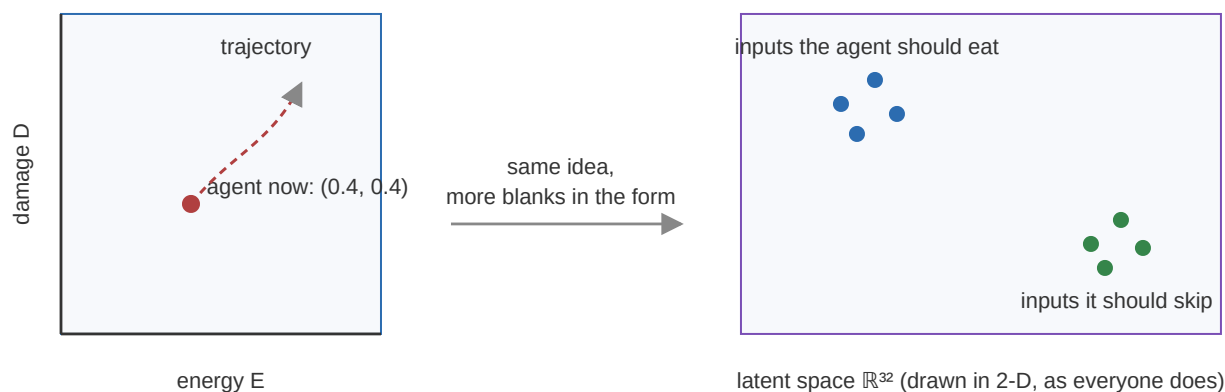
geometry and watches the head fail, it is doing surgery on one machine in the pipeline while holding the others fixed — which is what lets it say *where* a capability lives.

2.3 Spaces: anything you can describe with a list of numbers

The plane from algebra is the set of all pairs (x, y) — written $\mathbb{R}^2$. Nothing stops us using triples $\mathbb{R}^3$, or lists of 32 numbers, $\mathbb{R}^{32}$. A **point in $\mathbb{R}^d$ is just a completed form with d blanks**. You cannot picture 32 perpendicular axes, and neither can anyone else; working mathematicians picture 2 or 3 dimensions and let the algebra carry the rest. That is a legal and standard move, not a cheat.

Definition — state space, latent space

A **state space** is the set of all configurations a system can be in, described numerically. An agent whose situation is fully captured by (energy E , damage D) lives in a 2-dimensional state space: the square $[0,1] \times [0,1]$. A **latent space** (or embedding space) is a state space that a learned encoder invents for itself: in most of our papers, a 32-dimensional space where the encoder places each input as a point $z \in \mathbb{R}^{32}$. The entire "geometry of meaning" agenda is the study of *where the encoder chooses to put things* in that invented space.



Two spaces you will meet on nearly every page. Left: a literal state space — the agent's condition is a point, and its life is a curve through the square. Right: a latent space — an encoder has placed inputs as points, and (in a healthy geometry) inputs that *should be treated the same* have landed near each other. Whether that clustering happens, what organizes it, and whether it is load-bearing for behavior is the subject of roughly a third of the papers.

2.4 Functions of many variables, and the art of freezing

A function can take several inputs: $r(\text{color}, \text{label}, E)$ — reward as a function of what the item looks like, what it is, and the agent's current energy. Two reading skills:

- **Currying by freezing.** Fix all inputs but one and you get a one-variable function you can graph mentally. "Reward as a function of E , for this fixed item" is how you should read the allostatic-control paper's inverting reward: below the boundary b the reward is one thing; at $E \geq b$ it flips sign. One frozen slice at a time.
- **Piecewise definitions are just if-statements.** The notation $r = \text{base}(x)$ if $E < b$; $-\text{base}(x)$ if $E \geq b$ is literally `if E < b: return base(x) else: return -base(x)`. Papers in the control family are full of these, and they

read exactly like code.

In our papers — the homeostat's update rule is a function you already understand

The agency papers all share one little dynamical law (Chapter 14 develops it fully): each time step, energy decays and eating changes it — $E \leftarrow \text{clip}(E - 0.04 + \text{reward}, 0, 1)$, and the episode ends if $E \leq 0$. Read the arrow " $\leftarrow$ " as assignment, exactly as in code: the new E is computed from the old E. " $\text{clip}(\cdot, 0, 1)$ " pins the result into the interval $[0, 1]$. There is no mystery here; it is three lines of Python wearing mathematical clothing, and it defines the life-and-death stakes that make "concern" a meaningful word in these papers.

2.5 Indicator functions: turning yes/no into 0/1

One notation deserves special billing because the benchmark papers lean on it: $\mathbf{1}[\text{condition}]$ (a bold or blackboard 1) equals 1 when the condition is true and 0 when false. It converts logic into arithmetic so that *counting* becomes *summing*. "The fraction of test inputs where the model's answer doesn't change when you rotate the input" is written

$$\frac{1}{N} \sum_i \mathbf{1}[\text{argmax } f(g \cdot x_i) = \text{argmax } f(x_i)]$$

For each test input: score 1 if the prediction survived the transformation, else 0. Add up, divide by the count. An average of an indicator is a fraction — this is how "invariance score" formulas work in the learned-symmetry papers.

2.6 What you can now read

Type signatures $f: X \rightarrow Y$; compositions (encoder $\rightarrow$ head pipelines); $\mathbb{R}^d$ as "forms with d blanks"; state vs latent spaces; piecewise functions as if-statements; $\leftarrow$ as assignment; $\mathbf{1}[\cdot]$ as a truth-to-number converter; and the standard trick of reasoning about high dimensions through 2-D pictures.

CHAPTER 3

Exponents and Logarithms: The Mathematics of Doubling

If one chapter of high-school algebra had to carry this entire research program on its back, it would be this one. Logarithms appear in the definition of weakness, in description length and compression, in entropy and "bits," in loss functions, in the rate-distortion power laws, and in the log-log plots used to test them. The good news: everything reduces to one idea — **a logarithm counts doublings** — and one picture.

3.1 Exponents: repeated multiplication, and then more

You know $2^3 = 2 \cdot 2 \cdot 2 = 8$. The three rules worth re-memorizing, because the papers use them silently:

$$a^m \cdot a^n = a^{m+n} \quad (a^m)^n = a^{mn} \quad a^{-n} = \frac{1}{a^n}$$

Two extensions matter. First, **fractional exponents**: $a^{1/2} = \sqrt{a}$, and in general $a^{p/q}$ is a perfectly good number for any fraction. This is why an expression like $w(x)^{d/(d+2)}$ — the exponent in the program's reward-deformation law — is meaningful: with $d = 2$ it says $w^{1/2} = \sqrt{w}$, "resolution grows like the square root of how much you care." Second, **exponential decay**: 2^{-k} halves with each step of k . The MDL selector in the weakness benchmark scores each hypothesis as $2^{-\text{form_length}}$ — every extra symbol of description *halves* the hypothesis's prior credibility. That single minus sign in an exponent encodes the entire philosophy of Occam's razor.

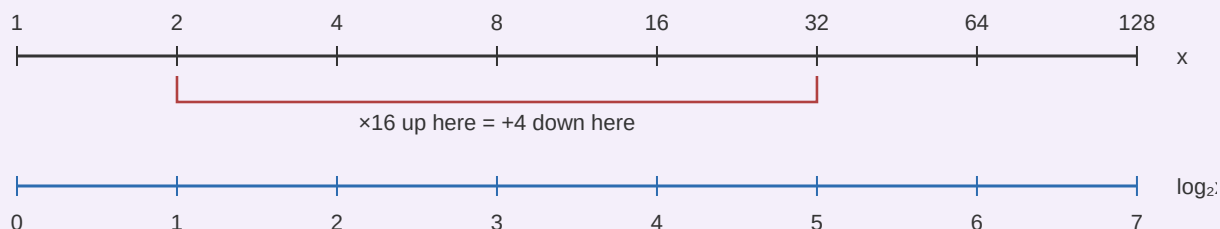
3.2 The logarithm is the question "how many doublings?"

Definition — logarithm

$\log_2(x)$ answers: "2 raised to what power gives x ?" Equivalently: how many times must you double 1 to reach x ? So $\log_2 8 = 3$, $\log_2 1024 = 10$, $\log_2 1 = 0$ (no doublings needed). The natural log $\ln(x)$ asks the same question with base $e \approx 2.718$; $\log_{10}$ asks it in "orders of magnitude." Changing base only rescales by a constant, which is why papers are often casual about which base they use.

Log answers grow *absurdly slowly*: to push $\log_2$ from 10 to 20 you must grow x from about a thousand to about a *million*. That slowness is the point. Logs are the tool for comparing quantities that differ by factors of thousands — like the number of inputs two hypotheses accept — on a single humane scale.

Visual proof — log turns multiplication into addition



The slide-rule fact. The top ruler is spaced so each doubling takes one equal step. Multiplying by 16 on the top ruler is *sliding four steps* on the bottom one. That is the identity $\log(ab) = \log a + \log b$, seen rather than derived: exponents add when you multiply powers, and logs *are* exponents. Engineers computed with physical versions of this picture (slide rules) for three centuries.

3.3 Bits: the log of a count is an amount of information

Suppose a friend thinks of one item out of 8 and you ask yes/no questions. With perfect halving you need exactly 3 questions: $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$. In general, distinguishing one option out of N requires $\log_2 N$ yes/no questions. This number is the **information content in bits**. It is why log-of-a-count appears all over the program:

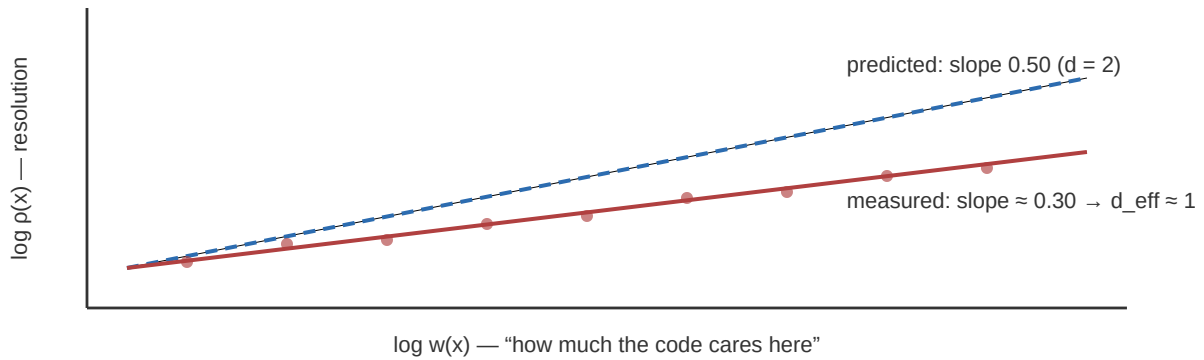
- **Weakness.** A hypothesis that accepts N possible worlds leaves $\log_2 N$ bits of freedom unspent. Strong claims spend bits; weak claims hoard them. The papers' "weakest compatible hypothesis" is the one that spends the fewest bits while still fitting the data.
- **Description length (MDL).** A program of length L symbols is one of roughly 2^L possible programs, so "short" and "a-priori probable" (2^{-L}) are the same statement read in opposite directions.
- **The Law of the Stack.** The autopoietic-control paper's inequality $w(\zeta_{i+1}) \leq 2^{w(\zeta_i)}$ says: a layer whose foundation has w bits of slack can express at most 2^w distinguishable configurations — each bit of freedom below doubles the room above. It is the 20-questions game stacked on itself.

3.4 Log-scale and log-log plots: how to read them, how they test laws

Plotting $\log y$ against $\log x$ performs a small miracle: **power laws become straight lines**. If $y = C \cdot x^\alpha$, take logs of both sides (using the slide-rule fact twice):

$$\log y = \log C + \alpha \cdot \log x$$

which is $Y = b + \alpha X$ — a line whose **slope is the exponent α** . This is not a stylistic choice; it is how one of the program's sharpest predictions is tested. The rate-distortion note derives, before any measurement, that a neural code with capacity limits should allocate resolution as $\rho^*(x) \propto w(x)^{d/(d+2)}$ — so for a 2-dimensional code the log-log slope should be $2/(2+2) = 0.5$. The measured slope came out ≈ 0.30 with tight bootstrap intervals — a straight line, but with the wrong tilt, implying the code behaves as if it were effectively *one*-dimensional. A falsified exponent is a far more interesting result than a vague "correlation exists," and it was readable directly off the tilt of a line.



A theory you can falsify with a ruler. On log–log axes, the power-law prediction and the data are both lines; the experiment is literally “compare the slopes.” The mismatch (0.30 measured vs 0.50 predicted) is itself informative: inverting $\alpha = d/(d+2)$ gives $d_{\text{eff}} = 2\alpha/(1-\alpha) \approx 0.9$ — the code concentrates its extra resolution along roughly one direction, not two.

Intuition pump — why nature keeps handing us power laws

A power law $y \propto x^\alpha$ is the signature of *scale-free trade-offs*: doubling the input always multiplies the output by the same factor (2^α), no matter where you start. Budget allocation problems — spend limited capacity where it matters most — have power-law solutions for the same reason a fair split of any pie looks the same at any zoom level. So when the program derives “resolution $\propto$ concern $^{d/(d+2)}$ ” from a capacity constraint, the surprising part is not the power law; it is the specific exponent, because the exponent remembers the *dimension* of the space being budgeted over. That is why measuring a slope can reveal a hidden dimensionality.

Pitfall — logs of small or zero counts

$\log(1) = 0$ and $\log(0)$ is undefined (nothing you raise 2 to gives zero). When a paper's quantity is log of a count, a value of 0 means “exactly one option survived,” not “nothing there.” And when you see +1 or + ϵ inside a log in code — $\log(x + 10^{-9})$ — it is a guard against the undefined $\log(0)$, not deep mathematics.

3.5 What you can now read

Fractional and negative exponents; 2^{-L} as “each extra symbol halves credibility”; $\log_2$ of a count as bits / 20-questions depth; the Law of the Stack inequality; why $\log(ab) = \log a + \log b$; and log–log plots as slope-reads-exponent instruments — including the program's own falsified-exponent result.

CHAPTER 4

Counting: Hypothesis Spaces and the Size of "All the Ways"

Much of the program's originality lives in an unfashionable branch of math: counting. The central quantity, weakness, is a *count*. The controls are counts. The impossibility arguments ("precision is capped at 1/3 because only 8 of 24 candidate angles are real") are counts. This short chapter provides all the combinatorics the papers use.

4.1 The multiplication principle

If a choice has a options and an independent second choice has b options, there are $a \cdot b$ combined options. That is the whole principle; everything else is applying it repeatedly.

- **Binary strings / possible worlds.** A world described by n yes/no features has $2 \cdot 2 \cdots 2 = 2^n$ possible configurations. The weakness-vs-simplicity toy experiment literally enumerates such worlds and counts how many each candidate rule accepts.
- **Boolean functions.** A rule assigning yes/no to each of N inputs is one of 2^N possible rules. With even 20 inputs that is about a million rules; with 256 inputs it exceeds the number of atoms in the observable universe. This double exponential is why "just try every hypothesis" is never an option and *selection principles* (simplicity? weakness?) are the real scientific question.
- **Experiment grids.** "36 cells = 4 conditions $\times$ 3 boundaries $\times$ 3 seeds" — the methodology sections are multiplication-principle arithmetic, nothing more.

4.2 Permutations and factorial growth

The number of ways to order n distinct things is $n! = n \cdot (n-1) \cdots 2 \cdot 1$ ("n factorial"), by the multiplication principle: n choices for first place, $n-1$ for second... The symmetric group S_n — all $n!$ shufflings of n items — appears in the weakness benchmark as one of the transformation families, and permutation *controls* (shuffle the labels, shuffle the pixels, re-run everything) appear throughout: if a result survives label-shuffling, it was never about the labels.

4.3 Counting as measurement: $|A|$ and fractions

Two recurring idioms:

$$\text{fraction} = \frac{|\{\text{items passing the test}\}|}{|\{\text{all items}\}|} \quad \text{normalized weakness} = \frac{W_G(f)}{|G|}$$

The left idiom is every accuracy, every recall, every "invariance score" in the papers: filter, count, divide. The right idiom rescales a raw count into $[0, 1]$ so different-sized transformation families can be compared — the neural weakness sweeps report *weakness_oracle_norm* for exactly this reason. When you meet an unfamiliar score, check first whether it is secretly a filtered count over a total. It usually is.

In our papers — a counting argument doing real scientific work

The neural-group-generator paper reports that encoder-based symmetry discovery seemed to fail with $F1 \approx 0.5$ — until a counting argument reframed it. The method scores 24 candidate rotation angles, of which only 8 are genuinely in the true group $\mathbb{Z}_8$. A scorer that (correctly) likes many angles can therefore never exceed precision $8/24 = 1/3$ at full recall — the *ceiling* was combinatorial, not a learning failure. Diagnosing that required no new experiment: only dividing 8 by 24. Cheap arithmetic, expensive insight — a pattern you will see repeatedly.

4.4 What you can now read

2^n configuration counts and 2^{2^n} -style hypothesis-space explosions; $n!$ orderings and permutation controls; $|A|/|B|$ fractions hiding inside every accuracy and score; and count-based normalizations like $W_G(f)/|G|$.

CHAPTER 5

Probability: The Grammar of Uncertainty

Every experiment in the repository lives in a noisy world: random seeds, random shocks, sampled training sets, stochastic policies. Probability is the bookkeeping system that keeps claims honest under that noise. You saw a little of this in your stats class; this chapter rebuilds it from scratch with the specific notation the papers use.

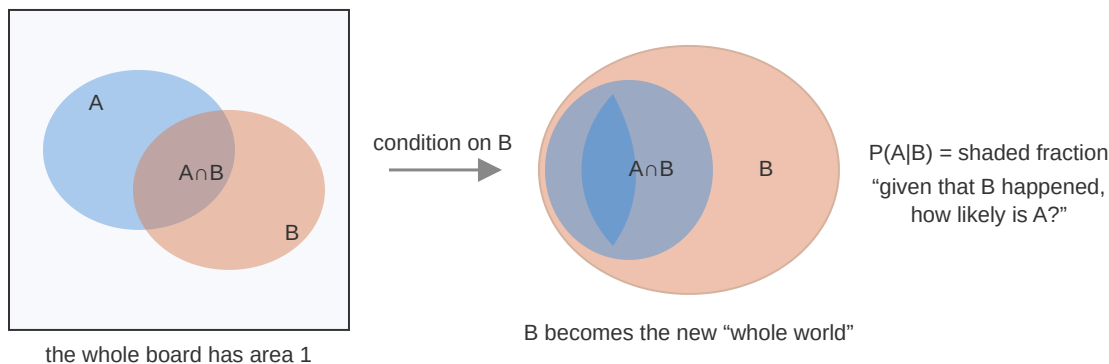
5.1 Probability is area

The cleanest mental model: imagine all the ways the world could turn out as a square dartboard of total area 1. An **event** is a region; its **probability** is its area. Everything else in probability theory is geometry on this board.

Definitions — the core vocabulary

$P(A)$ — the probability of event A; a number in $[0, 1]$. $P(A \cap B)$ ("A and B") — area of the overlap. $P(A \cup B)$ ("A or B") — area covered by either. $P(A | B)$ ("A given B") — the fraction of B's area that is also in A: the dart is known to have landed in B; how much of that region is A?

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$



Conditioning is zooming. $P(A | B)$ throws away everything outside B and re-measures A inside the new, smaller world. Every "given," every "conditional on," every vertical bar in the papers is this zoom operation.

This notation is live in the papers. The first-order-self experiments define their environment with lines like $P(\text{shock} | \text{food}) = 0.8$, $P(\text{shock} | \text{otherwise}) = 0.1$: "given that the agent just ate food, an external shock arrives with probability 0.8." One line of conditional probability fully specifies the hazard the agent must learn to attribute correctly.

5.2 Independence, and why "given" changes everything

Events A and B are **independent** if knowing B tells you nothing about A: $P(A | B) = P(A)$, equivalently $P(A \cap B) = P(A) \cdot P(B)$. Independence is an assumption you should notice papers *making* — "trials are drawn

independently," "seeds are independent" — because the statistics of Chapter 6 silently depend on it. The interesting science is usually in the *failures* of independence: the state-dependent-concern paper's whole point is that an agent's training data is *not* independent of its behavior (its policy steers which states it visits), and that coupling — not any lack of cleverness — is what makes certain reward rules unlearnable online.

5.3 Random variables, distributions, and $\mathbb{E}$

A **random variable** is a numerical quantity whose value depends on a random outcome — "the return of this episode," "the OOD accuracy of a model trained with this seed." Its **distribution** is the full list (or curve) of values it can take and how probable each is. Notation: $x \sim p$, "x is drawn from distribution p."

Definition — expected value

The **expected value** (mean) of a random variable is its probability-weighted average:

$$\mathbb{E}[X] = \sum_v v \cdot P(X = v)$$

Physically, $\mathbb{E}[X]$ is the *balance point* of the distribution — where the histogram would balance on a fingertip. It is a long-run average: the value your per-play results approach if you play forever. It is *not* a promise about any single play.

Intuition pump — expectation is a fair price

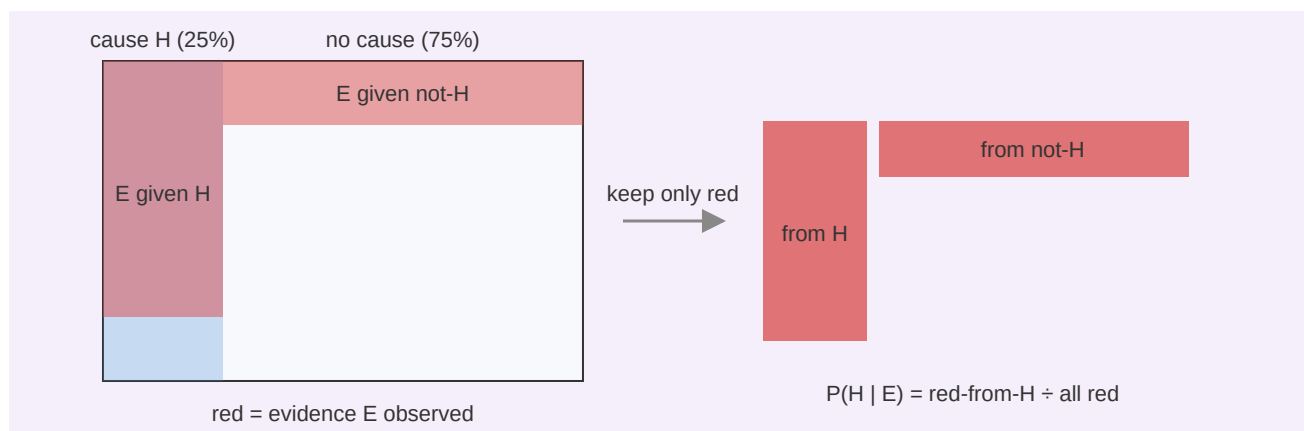
A ticket pays \$10 with probability 0.1 and \$0 otherwise. $\mathbb{E} = \$1$. The expectation is the *fair price* of the ticket: pay less and repeat forever, you profit; pay more, you bleed. Whenever a paper writes $\mathbb{E}[V(a) \mid \text{do}(h)]$ — the expected value of taking action a under an enforced condition — read: "the fair price of this action in this scenario." Agents in the program that plan by $\text{argmax}_a \mathbb{E}[\Delta E]$ are just refusing to overpay for actions.

Variance measures spread: $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$, the average *squared* distance from the mean. Its square root, the **standard deviation** σ , is back in the original units — "the typical wobble." The papers' tables report "mean $\pm$ std across seeds" — read the $\pm$ number as the typical seed-to-seed wobble, and be suspicious of any difference between conditions that is smaller than it. Several of our papers normalize quantities by $\sqrt{(\text{variance} + \epsilon)}$ — that is *rescaling to wobble units*, so that a dimension whose numbers are naturally 10 $\times$ larger doesn't drown out the others. (The vector-first-order-self paper found exactly this failure — one dimension's probe signal was anti-calibrated partly because of raw-scale mismatch — and per-dimension variance normalization is its prescribed fix.)

5.4 Bayes' rule: reversing the direction of "given"

Often you know $P(\text{evidence} \mid \text{cause})$ but want $P(\text{cause} \mid \text{evidence})$. The conversion is Bayes' rule, and it is one picture, not a formula to memorize.

Visual proof — Bayes' rule with areas



Bayes' rule is two zooms. Left: a population where the cause H is rare (thin blue column) but produces the evidence E reliably (tall red band), while the common no-cause world produces E occasionally (short red band). Right: condition on having seen E — collect all the red. The posterior $P(H | E)$ is just *which share of the red came from H* . Notice the punchline you may remember from stats folklore: even reliable evidence for a rare cause can leave the cause improbable, because the wide not- H region contributes a lot of red. Formally: $P(H | E) = P(E | H) P(H) / P(E)$ — but the picture is the proof.

In our papers — where Bayes shows up structurally

Beyond routine uses, Bayes-style reasoning appears at a load-bearing joint: the commitment-surface paper proves that the weakness selector is *Bayes-optimal* exactly when the transformation family used for scoring matches the family that actually generates deployment data (a uniform prior over the true group), and shows by counterexample that scoring against a *superset* group can invert the ranking. In primer terms: a prior is a bet about which worlds are likely; weakness is a good bet precisely when its implicit prior matches reality. "Whose prior is it?" turns out to be an experimental variable, not philosophy.

5.5 Distributions you will actually meet

Distribution	Shape	Where it appears in the program
Bernoulli / coin flip	two spikes: 1 with prob p , 0 with $1-p$	every pass/fail gate, every indicator; " $P(\text{shock} \text{food}) = 0.8$ " is a Bernoulli per step
Uniform	flat: all options equally likely	random seeds, uniform sampling of states in off-policy training, the "uniform prior over group elements" behind weakness optimality
Gaussian / normal	the bell curve, center μ , width σ	observation noise ($\sigma = 0.15$ in the agency environments), weight-noise perturbations in the repair experiments, Gaussian reward bumps in the metric-deformation work, and the shape that sample means always drift toward (see below)
Rademacher	± 1 with equal probability	random probe vectors in the Hutchinson sharpness estimator — a workhorse trick for peeking at giant matrices

One theorem to know by reputation: the **Central Limit Theorem**. Averages of many independent noisy things become bell-shaped, regardless of the noise's original shape, and their wobble shrinks like $1/\sqrt{n}$. This is why "average over more seeds" narrows error bars — and why halving your error bar costs *four times* the compute, a trade the Modal sweep budgets in the repo are constantly negotiating.

5.6 What you can now read

$P(A \mid B)$ as zoom; independence as "given changes nothing"; $x \sim p$; $\mathbb{E}$ as balance point / fair price; Var and σ as wobble (and $\sqrt{\text{var}+\epsilon}$ normalization); Bayes as red-area bookkeeping; Bernoulli/uniform/Gaussian by role; and the $1/\sqrt{n}$ law behind every error bar.

CHAPTER 6

Statistics: How the Papers Earn the Right to Their Claims

Your stats class probably taught you t-tests and p-values. Here is a surprise: **this research program almost never uses them.** Its statistical style is different and, once seen, arguably more honest: pre-registered numeric *gates*, effect sizes with *bootstrap confidence intervals*, multiple random *seeds*, and batteries of *controls designed to fail*. This chapter teaches exactly that toolkit — it is the key to reading every results section in the repository.

6.1 Sample vs population, and the $1/\sqrt{n}$ law

An experiment trains, say, 3 models with 3 random seeds and reports the mean. The *population* is the (imaginary, infinite) set of all models you could have trained; the 3 you got are a *sample*. The sample mean wobbles around the true mean, and the size of that wobble — the **standard error (SE)** — shrinks like $\sigma/\sqrt{n}$. Three seeds give you a rough fix; the repo's larger sweeps (64 seeds, 256 models, 1000-trial benchmarks) buy narrower error bars at $1/\sqrt{n}$ prices. When a paper honestly writes "only 3 seeds — wide error bars," this law is what it is apologizing to.

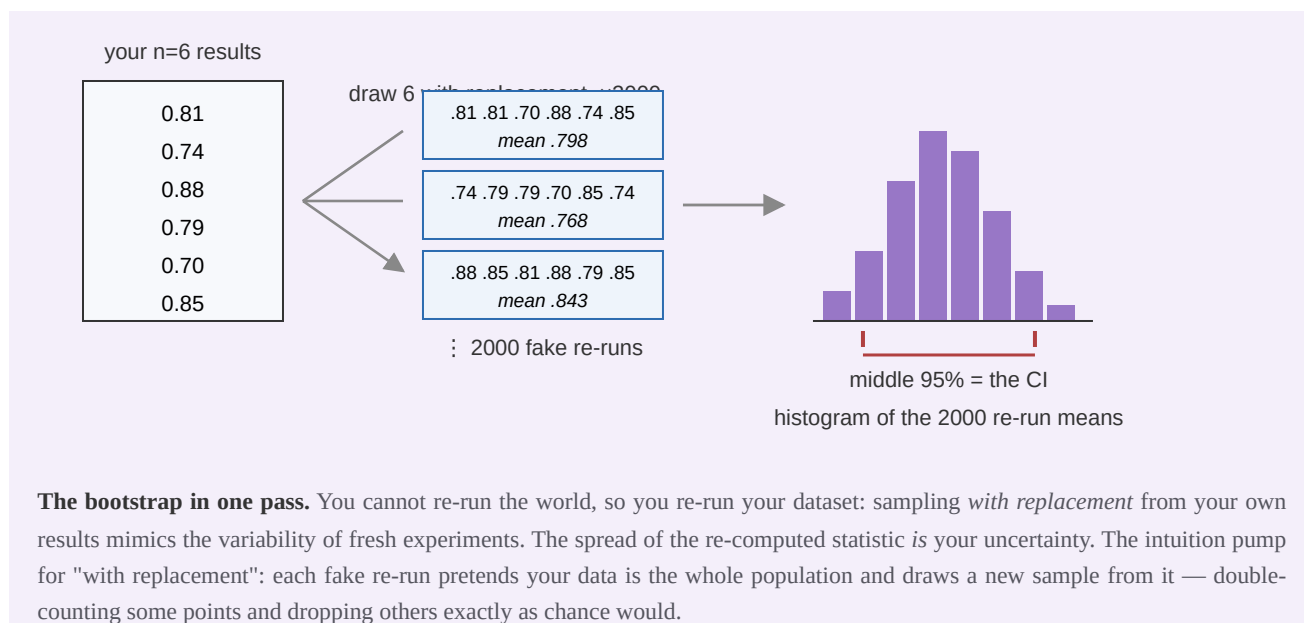
6.2 The bootstrap: error bars without formulas

Classic statistics derives error bars from formulas that assume bell-curve behavior. The repository's papers mostly use a more computational, more robust idea due to Efron: **ask the data itself.**

Definition — bootstrap confidence interval

Given n measurements: (1) draw n values *from your own data, with replacement* (so some values repeat, some are skipped); (2) compute the statistic (mean, ratio, whatever) on this fake "re-run"; (3) repeat thousands of times; (4) the middle 95% of those re-computed statistics is the **95% bootstrap confidence interval**. The repo's implementation is literally this recipe with 1,000–2,000 resamples and a fixed seed.

Visual proof — resampling simulates re-running the experiment

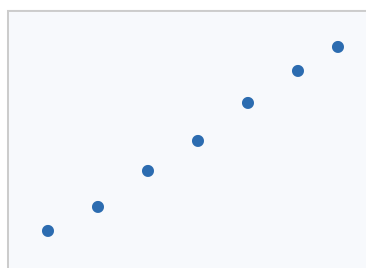


How the papers use the interval: the standard gate is "95% CI lower bound > 0." Read: even in the unluckiest plausible re-run, the effect stays positive. The long-horizon-bottleneck paper offers a perfect specimen of the gate doing its job: one condition measured specificity +0.695 — a positive mean! — but its CI was $[-0.376, 2.080]$, crossing zero, so the paper reported it as a *negative result*. The mean flattered; the interval told the truth.

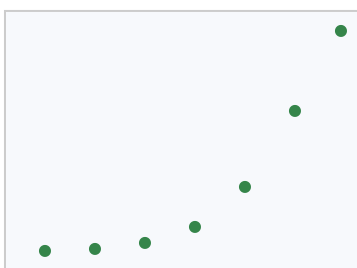
6.3 Correlation: Pearson, Spearman, and the difference that keeps mattering

Definitions

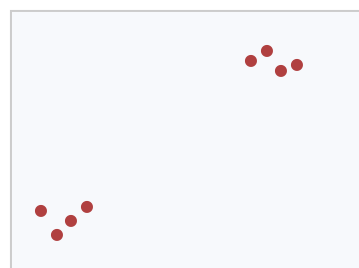
Pearson r measures *linear* co-movement between paired quantities: $r = +1$ (perfect line up), 0 (no linear relation), -1 (perfect line down). **Spearman ρ** is Pearson's r computed on the *ranks* (1st, 2nd, 3rd...) instead of the raw values: it asks only "when one goes up, does the other?", ignoring scale and tolerating curves.



$r \approx 1, \rho \approx 1$: linear



curved: $\rho = 1, r < 1$



two clumps: r high, ρ weak inside

Why the papers report both. Right panel is the trap the costly-null-probes paper fell into and documented: its probe signal had Pearson $r = +0.84$ with true uncertainty — driven entirely by one big clump being higher than the other — while Spearman ρ within the fine structure was +0.21: the probe couldn't actually rank *which* situations deserved inquiry. A later paper (current-error calibration) fixed the mechanism and flipped a ρ of -0.55 ("anti-calibrated": reliably pointing the *wrong* way) to +0.62. Watch for the sign and the kind of correlation; both carry scientific content here.

6.4 Regression, residuals, and "controlling for"

Least-squares regression fits the line (or plane) $\hat{y} = b + \beta_1 x_1 + \beta_2 x_2 + \dots$ that minimizes squared errors. Each coefficient β reads: "holding the other variables fixed, one unit more of this variable predicts β more of y ." The **residual** is what's left, $y - \hat{y}$ — the part the predictors can't explain.

Residuals power a maneuver the papers use to fight confounding: **partial correlation**. To ask "is weakness related to OOD accuracy *beyond* what augmentation explains?", regress both weakness and OOD on the augmentation variable, keep the two residual series, and correlate *those*. If the residual correlation survives (the flagship weakness paper reports a residual effect of $\beta = +0.365$ this way), the relationship is not just the confound wearing a costume. The same residual trick, run in reverse, produced one of the program's cleanest negative results: a topology-generalization correlation of 0.208 collapsed to 0.004 once a "seam" variable was controlled for — the topology was a passenger, not a driver.

6.5 z-scores: putting different rulers on one scale

$$z = \frac{x - \text{mean}}{\text{std}}$$

"How many typical-wobbles above or below the group average is this value?"

z-scoring appears whenever the papers compare things measured in incompatible units. The long-horizon-bottleneck suite z-scores the four memory-slot sensitivities *within each model*, then defines its primary statistic as $z(\text{critical slot}) - \text{mean } z(\text{other slots})$ — "how many wobbles more sensitive is the slot that *should* matter, compared to its siblings' range. The same idea, applied across time instead of across slots, is the scale-normalization fix of the V-probe papers: divide each dimension's signal by $\sqrt{\text{its own running variance}}$ so a naturally loud channel cannot drown a quiet one.

6.6 The house methodology: gates, seeds, controls, oracles

Now the centerpiece. Nearly every experimental paper in the repo follows one template, and once you know it, results sections become fast reading:

1. **Pre-registration.** Before running the sweep, the authors freeze (often in a committed file) the exact numeric thresholds — "G2: mediated MAE reduction $\geq 25\%$," "G14: Spearman $\rho \geq 0.5$ " — plus an *interpretation matrix* saying in advance what each pass/fail combination will be taken to mean. This forecloses the temptation to move goalposts after seeing data.
2. **Gates.** Each claim is tied to one or more binary gates. A "pass" is a pre-committed threshold being met; a "fail" is reported with the same prominence. Several papers' headline contributions *are* honest gate failures (the falsified rate-distortion exponent; the ensemble whose variance refused to track error; the probe that fired everywhere).
3. **Seeds.** Every random choice (initialization, sampling) is controlled by a recorded seed; conditions are re-run across several seeds (3 for cheap sweeps, up to 64 for statistical polish) and reported as mean $\pm$ std or with

bootstrap CIs. A result that holds at one seed and dies at another is noise, and the repo treats it as such.

4. **Controls designed to fail.** The signature move. Alongside the real condition run: *wrong-signal* variants (feed the mechanism garbage of the right shape), *stale* variants (right signal, out of date), *shuffled* variants (right data, scrambled pairing), *matched-random* variants (random behavior with the same budget — e.g. a random prober granted exactly as many probes as the learned one chose to use). The logic: if a control that *shouldn't* work works anyway, your metric is measuring something other than what you think. A learned mechanism only gets credit for beating its matched controls.
5. **Oracles as ceilings.** Conditions with cheat access to ground truth (oracle-source labels, oracle boundary features) bound what is achievable. "Learned reaches 90% of oracle" is a strong sentence; "learned beats random but sits at 30% of oracle" tells you how much room remains.

Intuition pump — gates are unit tests for claims

If you have ever written software tests, you already understand this methodology. A pre-registered gate is a unit test written *before* the code: it pins down, numerically and in advance, what "working" means. Controls are the adversarial test cases — the inputs on which the system *should* fail, where a pass reveals a bug in the test itself. And the repo's insistence on publishing failed gates is just the rule that you don't delete red tests from the CI log. Reading a results table is code review, and by the end of this book you will do it fluently.

6.7 Effect sizes: the number that matters more than significance

A difference can be statistically real and practically boring. The papers therefore lead with **effect sizes** — magnitudes in meaningful units: percentage-point lifts ("+51.5 pp OOD accuracy from oracle augmentation"), ratios ("selectivity 16.9× — the agent probes the affected buckets almost seventeen times more densely than unaffected ones"), percent reductions ("false self-credit down 82%"), and normalized gap closures ("9.6% of the oracle gap closed" — a polite way to say the mechanism barely helped). When you read a table, find the effect size first, the interval second, and only then the pass/fail marks.

Pitfall — one word, two meanings: "AUC"

In mainstream machine learning, AUC usually means area under the ROC curve (a ranking-quality score where higher is better, 0.5 is chance). Some of this repo's agent papers use "AUC" for something else: the *area under the error-vs-time learning curve* — total error accumulated over an episode window, where *lower* is better. And a third relative, "rank AUC," appears in the real-model validation work: the probability that a random positive example outranks a random negative one (ties counting $\frac{1}{2}$) — that one is the mainstream ranking sense. Check which is in play before interpreting a number; the papers define it locally every time.

6.8 What you can now read

Mean $\pm$ std across seeds; SE and the $1/\sqrt{n}$ law; bootstrap CIs and the "CI lower bound > 0 " gate; Pearson vs Spearman (and anti-calibration as negative ρ); regression coefficients, residuals, and partial correlation as "controlling for"; z-scores and specificity statistics; the gate/seed/control/oracle template; effect sizes as the headline; and the three local meanings of "AUC."

PART II

The Geometry of Meaning: Linear Algebra

The program's founding observation is that adaptive systems keep reaching for geometric language. To read the papers, you must be able to *do* a little of that geometry: vectors, angles, projections, and the machinery of matrices. This part builds all of it from the Pythagorean theorem up. Nothing beyond Algebra 2 and SOH-CAH-TOA is assumed.

CHAPTER 7

Vectors, Dot Products, and Cosine Similarity

7.1 A vector is an arrow and a list — at the same time

A **vector** is a list of numbers, $v = (3, 2)$, that you may equally picture as an arrow from the origin to the point $(3, 2)$. Both views are always available and you should toggle freely: the *list* view is how computers store embeddings; the *arrow* view is how humans reason about them. Addition stacks arrows tip-to-tail ($u + v$); scaling stretches them ($2v$); and the **norm** (length) comes straight from Pythagoras:

$$\|v\| = \sqrt{v_1^2 + v_2^2 + \dots + v_d^2}$$

The distance between two points is the norm of their difference, $\|u - v\|$ — the "L2 distance" or Euclidean distance. When the long-horizon-bottleneck papers measure how much a model's final hidden state moves when one early clue is flipped, the measurement is literally $\|state_{flipped} - state_{original}\|$: Pythagoras in 2,048 dimensions.

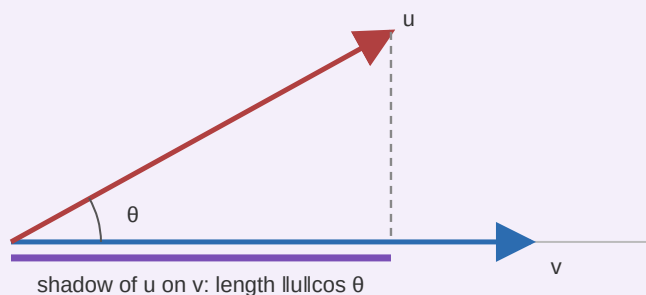
7.2 The dot product: multiply-and-add with a geometric soul

Definition — dot product

$$u \cdot v = u_1v_1 + u_2v_2 + \dots + u_dv_d \quad \text{and the miracle:} \quad u \cdot v = \|u\| \|v\| \cos \theta$$

The left side is arithmetic a spreadsheet can do; the right side says the same number equals the product of the lengths times the cosine of the angle between the arrows. One computation, two readings — and the second one turns raw number-lists into *angles*, i.e. geometry.

Visual proof — the dot product is a shadow



$\cos \theta = 1 \rightarrow$ same direction

$\cos \theta \approx 0.9 \rightarrow$ "similar"

$\cos \theta = 0 \rightarrow$ unrelated (perpendicular)

$\cos \theta = -1 \rightarrow$ opposite

cosine similarity =
shadow length $\div$ arrow length

Projection. Drop a perpendicular from u 's tip onto v 's line: the shadow has length $\|u\|\cos \theta$ (that is SOH-CAH-TOA — adjacent = hypotenuse $\times$ cos). So $u \cdot v = (\text{shadow of } u \text{ along } v) \times (\text{length of } v)$: the dot product measures *how much of u points the way v points*. Positive: broadly agreeing directions. Zero: perpendicular, no shared component. Negative: opposed.

7.3 Cosine similarity: the workhorse of the embedding papers

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \|v\|} \in [-1, 1]$$

Dividing out the lengths leaves the pure angle: **cosine similarity** compares directions while ignoring magnitudes. This is the single most used measurement in the geometry papers, because embedding vectors' lengths are often uninformative while their directions encode meaning. Concretely: the learned-symmetry papers score a candidate rotation by the cosine between a rotated image's pixels and other same-class images; the concept-geometry probes ask whether "attractor" and "constraint" vectors point in related directions; and centered-cosine comparisons underlie the program's favorite representation metric, which deserves its own box:

In our papers — the cluster gap, decoded symbol by symbol

$$gap_a = \text{mean } \cos(z_i, z_j) \text{ over pairs with } a_i = a_j - \text{mean } \cos(z_i, z_j) \text{ over pairs with } a_i \neq a_j$$

Piece	Meaning
z_i	the encoder's latent vector for input i — first <i>centered</i> (subtract the average vector, so the shared background direction doesn't inflate every similarity) and unit-normalized
a_i	the label along the axis being tested: same reward class? same color? same paraphrase orbit?
<i>first mean</i>	average similarity of pairs that <i>should</i> go together
<i>second mean</i>	average similarity of pairs that shouldn't
<i>gap</i>	how much more tightly the encoder groups same-class inputs than different-class ones

Reading the numbers: $gap \approx 0$ means the axis is not organized at all; the agency papers report gaps like +1.84 or +1.96 (on a scale where $\approx +2$ is the ceiling, since similarities live in $[-1, 1]$) for a fully reward-organized encoder. The valence-object-formation headline — `reward_gap +1.96` while `color_gap +0.005` — is this one formula, run twice with different labelings, saying: *the encoder built its world out of what mattered to it and threw away how things looked.*

7.4 Directions as concepts; projection as surgery

If same-class latents cluster, their *average direction* is a usable handle: a "concept direction" u (computed in the papers as the within-class mean of centered vectors, unit-normalized). Two operations follow:

- **Reading:** $z \cdot u$ measures how much of concept u is present in latent z (the shadow again).
- **Ablating (causal surgery):**

$$z_{\text{ablated}} = z - (z \cdot u) u$$

subtract off exactly the u -shadow, leaving the component of z perpendicular to u . If behavior depending on that concept survives, the direction wasn't load-bearing; if behavior collapses, it was. This one-line operation powers the passive-to-active geometry result: ablating the paraphrase direction barely hurt a passively-trained model (specific drop +0.07) but devastated the action-coupled one (+0.49, a $\sim 7\times$ ratio) — the geometry had gone from decoration to load-bearing wall. The crucial control: subtracting a *random* direction of the same size, which should (and did) hurt much less.

Pitfall — a real self-correction from the planning papers

The ablation tool also produced one of the program's honest reversals: in the planning-hardening experiments, ablating the *reward* direction hurt exactly as much as ablating a random direction (−0.49 vs −0.52). Conclusion: there, concern was not stored along any single axis — it was a *distributed* pattern, and "the reward axis" was a fiction of the analysis. Same tool, same rigor, opposite verdict. Directions are hypotheses, not guarantees.

7.5 One-hot vectors and feature lists

Two humble encodings appear everywhere. A **one-hot** vector represents a category as all zeros with a single 1 — action "consume" among {skip, consume, null} becomes (0, 1, 0); it is how discrete choices enter numeric machinery. And a **feature vector** is any hand-built list: the agency papers feed heads the list $[E, \sin(\pi E), \cos(\pi E), \sin(2\pi E), \cos(2\pi E), \sin(4\pi E), \cos(4\pi E)]$ — the scalar energy plus sine/cosine waves of increasing frequency (your trig class, employed at last). Why: smooth networks struggle with sharp changes; giving them wavy inputs of several frequencies lets them build sharp-ish functions of E by mixing waves, Fourier-style (Chapter 13 revisits this).

7.6 What you can now read

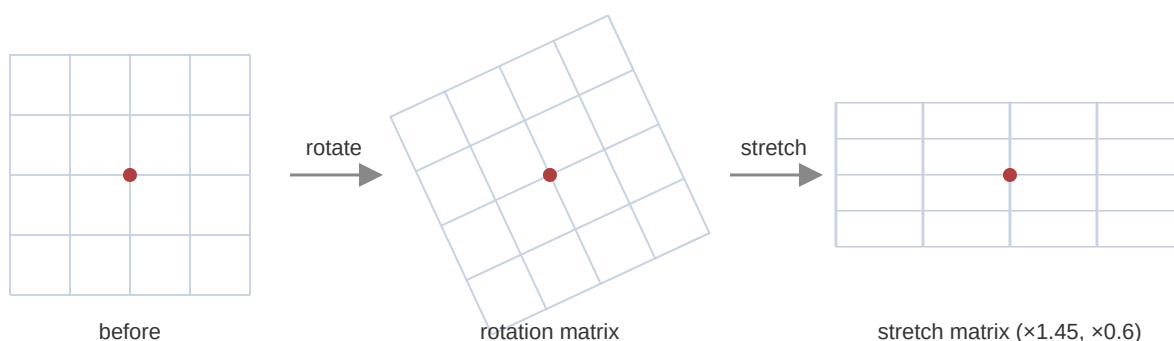
$\|v\|$ and L2 distance; $u \cdot v$ both as multiply-add and as $\|u\|\|v\|\cos \theta$; cosine similarity and centered cosine; the cluster-gap formula; concept directions; projection $(z \cdot u)u$ and ablation $z - (z \cdot u)u$; matched random-direction controls; one-hot and Fourier feature encodings.

CHAPTER 8

Matrices: Transformations, Spectra, and the Shape of a Cloud

8.1 A matrix is a function that respects addition

A **matrix** is a grid of numbers, but the productive view is: *a machine that takes a vector and returns a vector*, built entirely from multiply-and-add. The matrix-vector product Av computes one dot product per row of A : each output coordinate is a weighted blend of the input coordinates. Every layer of every neural network in the papers is a matrix (plus a bias and a squashing function). Geometrically, matrices are exactly the transformations that keep grid lines straight, parallel, and evenly spaced: rotations, stretches, shears, projections, and combinations — no bending.



Matrices move whole spaces. Grid lines stay straight and parallel — that is the defining property ("linearity"). Composition of matrices is matrix multiplication: do one, then the other.

8.2 Eigenvectors: the directions a matrix leaves alone

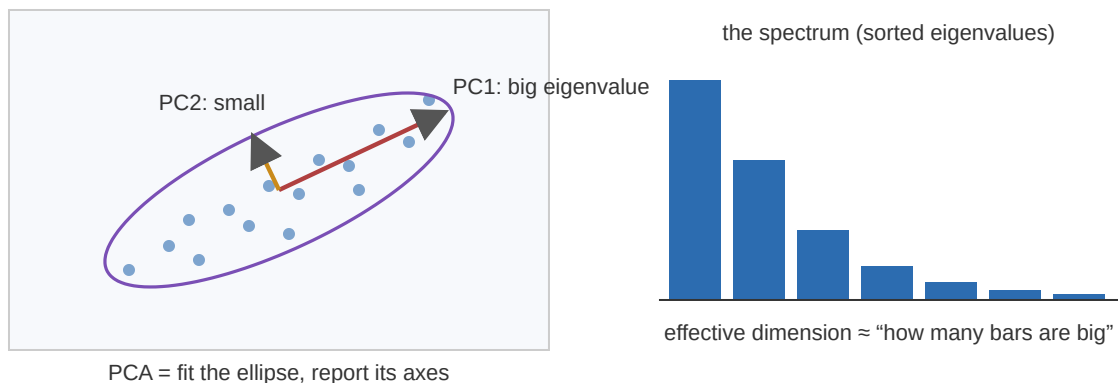
Definition — eigenvector, eigenvalue

An **eigenvector** of matrix A is a nonzero direction v that A does not rotate, only rescales: $Av = \lambda v$. The rescaling factor λ is its **eigenvalue**. Eigenvectors are a transformation's "grain" — the axes along which its action is pure stretch — and the list of eigenvalues (the **spectrum**) summarizes how strongly it acts along each grain.

Why care: repeated processes amplify their largest-eigenvalue direction (this is why feedback loops "find" their dominant mode); and symmetric matrices — including every covariance matrix — have a full set of perpendicular eigenvectors, which makes the next section possible.

8.3 Covariance and PCA: the ellipse around a data cloud

Take a cloud of points (say, 500 latent vectors). Its **covariance matrix** records, for every pair of coordinates, how they co-vary. The magic: the covariance matrix's eigenvectors are the cloud's *natural axes* — the directions of greatest, second-greatest, ... spread — and its eigenvalues are the variances along them. Fitting this ellipse and reading off its axes is **Principal Component Analysis (PCA)**: the standard way to find the few directions along which a high-dimensional cloud actually varies, and the tool behind every 2-D "latent space visualization" figure in the papers.



The shape of a cloud is a spectrum. Left: the covariance ellipse and its eigen-axes. Right: sorted eigenvalues. A cloud living in 32 coordinates might have only ~ 2 large eigenvalues — then it is "effectively 2-dimensional." The papers make this precise with **effective rank** / **effective dimension** / **participation ratio**: entropy-flavored counts of how many eigenvalues carry real weight. This is the d_{eff} in the rate–distortion law of Chapter 13, and the "collapse" diagnostics in the embedding experiments (a healthy space uses many directions; a collapsed one squeezes onto few).

8.4 SVD and low rank: every matrix is stretch-between-rotations

The **singular value decomposition (SVD)** says any matrix — square or not — factors as (rotation) $\times$ (pure stretch along perpendicular axes) $\times$ (rotation). The stretch factors, the **singular values**, generalize eigenvalues and are always ≥ 0 . Counting how many are meaningfully nonzero gives the matrix's **rank** — how many independent directions it genuinely uses.

Three program uses. (1) *Finding axes*: the planning-hardening papers build a matrix from per-class mean latents and take its top singular vector as "the concept axis" to ablate. (2) *Low-rank updates* / *LoRA*: fine-tuning a huge model can be done by adding a matrix that is deliberately rank-limited (expressible with few singular values) — cheap, and used in the commitment-surface experiments on real language models; a related trick adds a **rank-one** matrix $v v^T$ (an outer product — the opposite of a dot product: column times row giving a whole matrix) to a similarity kernel to implant a value-weighted bump, which is precisely the "metric deformation" operation in the phase-6 real-model experiments. (3) *Spectral-mass normalization*: some patching experiments remove "the smallest subspace explaining 50% of the spectral mass" — i.e., the fewest stretch-axes that carry half the total stretching — so that interventions of different sizes become comparable.

8.5 The Jacobian: the local matrix of any smooth map

A nonlinear function $\mathbb{R}^d \rightarrow \mathbb{R}^k$ bends grids — but zoom in far enough and any smooth map looks linear. The matrix it locally resembles is its **Jacobian** $J(x)$: entry (i, j) says how fast output i responds to a nudge of input j , at the point x . Chapter 11 builds this from derivatives; here, one preview matters. From the Jacobian of an encoder you can compute $g(x) = J(x)^T J(x)$ — the **pullback metric** — which answers: *if I move a tiny step in input space, how far does my latent point move?* Directions the encoder stretches are directions it treats as important; $\sqrt{\det g}$ measures the local "map scale." This is the mathematical heart of the metric-deformation papers, and Chapter 9 tells that story properly.

8.6 What you can now read

Av as blend-of-columns; matrices as straight-line-preserving transformations; eigenvectors/eigenvalues as a transformation's grain; covariance ellipses, PCA, spectra, and effective dimension/rank; SVD, singular values, low-rank and LoRA, rank-one outer-product bumps; and the Jacobian with its pullback metric $J^T J$.

CHAPTER 9

Metrics, Curvature, and Gauge: When Geometry Becomes Physics

The program's boldest geometric claim is that a system's *values* literally reshape the geometry of its internal space — that concern "warps the metric" the way mass warps spacetime in general relativity. That sentence sounds like a metaphor. This chapter shows it is a computation, and gives you the three ideas needed to read the metric-deformation and gauge-transport papers: what a *metric* is, what *curvature* and *geodesics* are, and what a *gauge* is and why it is a threat to scientific claims.

9.1 A metric is a local ruler that can vary from place to place

In flat space, distance is the fixed Pythagorean formula. But imagine a map where one inch near the center represents one mile, while one inch at the edge represents ten. To measure a real journey you must carry a *ruler that changes as you move*. That position-dependent ruler is a **metric** — written $g(x)$, a small matrix at each point x telling you how to convert tiny coordinate steps into real distances there.

Definition — pullback (induced) metric

When an encoder ϕ maps inputs to latent vectors, it induces a metric on input space via its Jacobian (Chapter 8):

$$g(x) = J_{\phi}(x)^T J_{\phi}(x)$$

Read it as: "to measure a step near x , first see how much the encoder *stretches* that step (J moves it into latent space), then measure it there." Where the encoder stretches inputs a lot, small real differences map to big latent differences — the metric there is "large," meaning the system resolves fine distinctions. Where it compresses, distinctions blur. The number $\sqrt{\det g(x)}$ is the local *volume scale* — how much latent room the system spends per unit of input — and it is the quantity the rate–distortion papers actually measure.

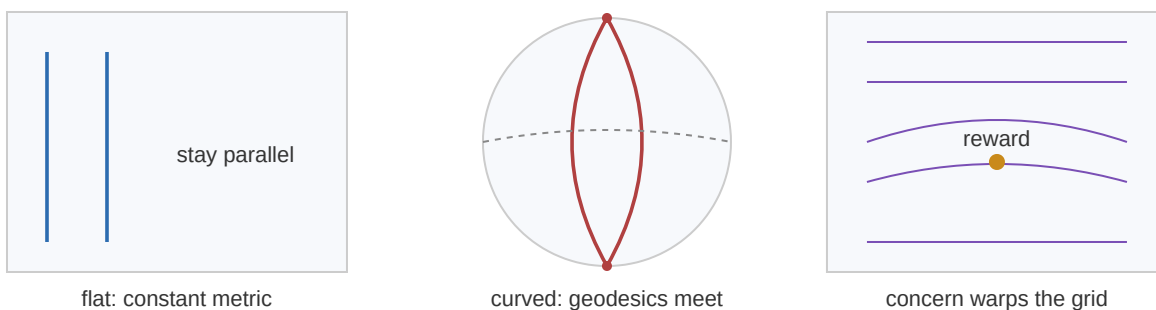
Intuition pump — the encoder decides where to spend resolution

A camera with a fixed number of pixels must choose what to render sharply. Point it at a face and it can spend pixels on the eyes (high metric there — small facial differences become large image differences) or spread them evenly (flat metric). A learned encoder faces the same budget. The program's central geometric hypothesis is that a *concern signal* (reward, value) tells the encoder where to spend: important regions get a magnified local ruler. "Concern deforms the metric" means precisely "the places the system cares about are the places where its internal ruler grows, so it can tell nearby important things apart." That is a measurable prediction about $\sqrt{\det g}$, not a poem.

9.2 Geodesics and curvature: straight lines and their failure to stay parallel

Once distance varies from place to place, "straight line" must be redefined as **shortest path** — a **geodesic**. On a globe, the shortest flight from New York to Madrid bows north; it is "straight" in the only sense the curved surface allows. **Curvature** is the intrinsic measure of how much a space departs from flatness, and it has an operational signature you can feel: on a curved surface, two geodesics that start parallel do not stay parallel (lines of longitude

are parallel at the equator but meet at the poles). A space whose metric $g(x)$ varies from point to point is generally curved; a space with constant g is flat.



From flat to warped. Left: constant ruler, parallel geodesics — flat space. Middle: a sphere, where "straight" paths converge — curvature. Right: the program's picture — a reward bump swells the local metric near what matters, warping the grid so the system's internal distances devote more room to high-value regions. General relativity uses the identical mathematics with mass in place of reward; the papers borrow the framework deliberately.

9.3 The reward-deformation law, previewed

Putting the ruler idea together with a budget produces the program's sharpest geometric prediction. If a system has limited capacity (finite total resolution) and wants to minimize *value-weighted* error, then — as Chapter 13 derives with a little calculus — the optimal resolution density obeys a power law:

$$\rho^*(x) \propto w(x)^{d/(d+2)}$$

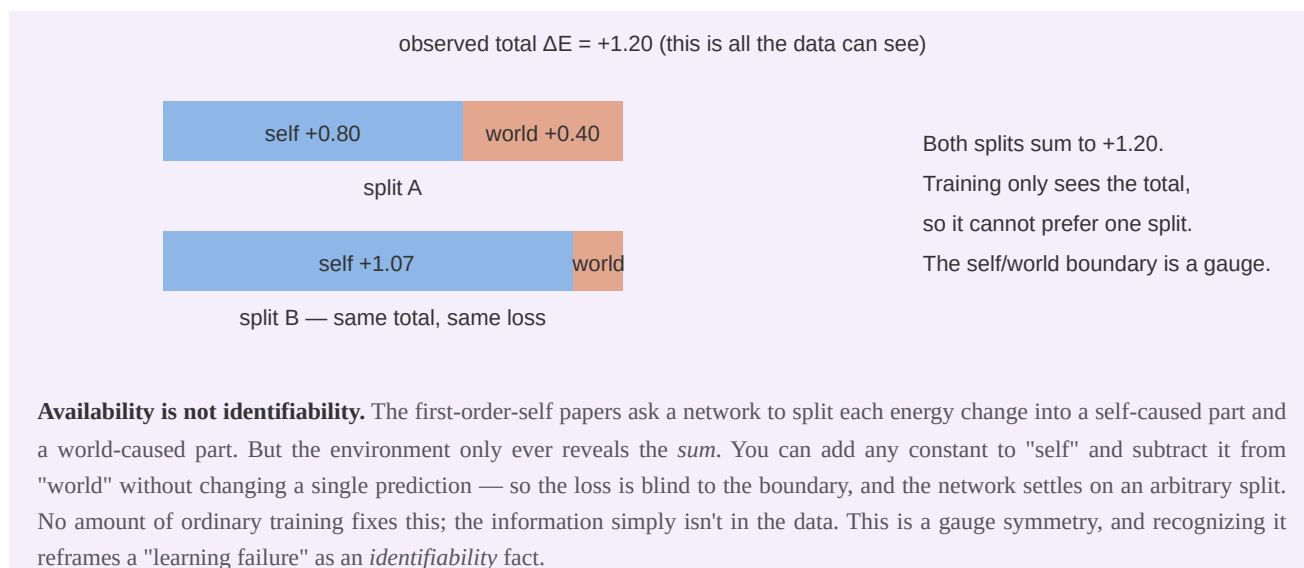
Resolution you spend at x grows as concern $w(x)$ raised to the power $d/(d+2)$, where d is the space's (effective) dimension.

You already have every piece: ρ is $\sqrt{\det g}$ (Section 9.1), w is concern (a nonnegative weighting, Chapter 2), the exponent is a fraction (Chapter 3), and it is tested by a log–log slope (Chapter 3). The headline empirical result — measured slope ≈ 0.30 against a predicted 0.50, implying $d_{\text{eff}} \approx 1$ — is a falsified exponent that told the researchers their code was concentrating resolution along essentially one direction. A metaphor cannot be falsified to three decimal places; this framework can, and was.

9.4 Gauge: the freedom that isn't real, and why it endangers claims

Here is the subtlest and most important idea in the program's theoretical papers. A **gauge symmetry** is a way you can change your *description* of a system without changing anything *observable*. Physics is full of these — you can add a constant to all altitudes and no dropped ball falls differently, because only altitude *differences* are physical. The choice of zero is a gauge; it is convention, not content.

Visual proof — the self/world gauge in the agency papers



The fix is the same one physics uses: **break the gauge with an intervention**. If the agent can *do nothing* on purpose (the "null action"), it observes the world's contribution alone, which pins down the previously-free constant. In symbols the papers use: $self_ \Delta E = \Delta E_{observed} - \Delta E_{null}$. The null-intervention paper reports this cut false self-credit by 82%. Chapter 18 develops interventions in full; the point here is geometric-conceptual: **a gauge is descriptive freedom, and only an intervention — not more passive data — can remove it**.

In our papers — the gauge-fixed concern transport theorem

The gauge-fixed-concern-transport paper stacks all three ideas of this chapter into one inequality. A distinction an agent draws is only genuinely "load-bearing" if (1) it carries enough *concern mass* W (weighted count of situations where it matters — Chapter 19), (2) that mass *survives transport* across context changes with bounded loss, and (3) after *fixing the gauge* with an intervention, the distinction still changes the decision by at least ϵ (the "commitment effect" CE). The theorem:

$$Load_{\gamma}(h) \geq (\alpha - \delta) \cdot \epsilon$$

In words: load-bearing-ness is at least (starting concern mass α , minus transport loss δ) times the gauge-fixed commitment effect ϵ . Every symbol is something this book has now built: a weighted count, a bounded loss, an intervention effect. The paper validated it on a 320-run GPU suite where a distractor probe scored a near-perfect AUC of 0.999 yet had a causal patch-effect $\sim 36\times$ smaller than the real feature — a vivid demonstration that decodability (a probe can read it) is not use (it drives the decision), which is the whole reason the gauge-fixed, intervention-based definition is needed.

9.5 What you can now read

Metrics $g(x)$ as position-varying rulers; the pullback metric $J^T J$ and $\sqrt{\det g}$ as resolution/volume scale; geodesics as shortest paths and curvature as parallel-lines-that- converge; the reward-deformation power law $\rho^* \propto w^{d/(d+2)}$ and its falsified exponent; gauge symmetry as descriptive freedom (the self/world split); gauge-fixing by intervention; and the $Load \geq (\alpha - \delta) \cdot \epsilon$ bridge theorem.

PART III

Change, Optimization, and Information

Learning is optimization, and optimization is calculus. This part builds the calculus the papers use — rates of change, gradients, the descent algorithm that trains every network in the repo, the integrals and Lagrange multipliers behind the theoretical results, and the information theory that measures it all. You need no prior calculus; we build the derivative from the idea of a slope.

CHAPTER 10

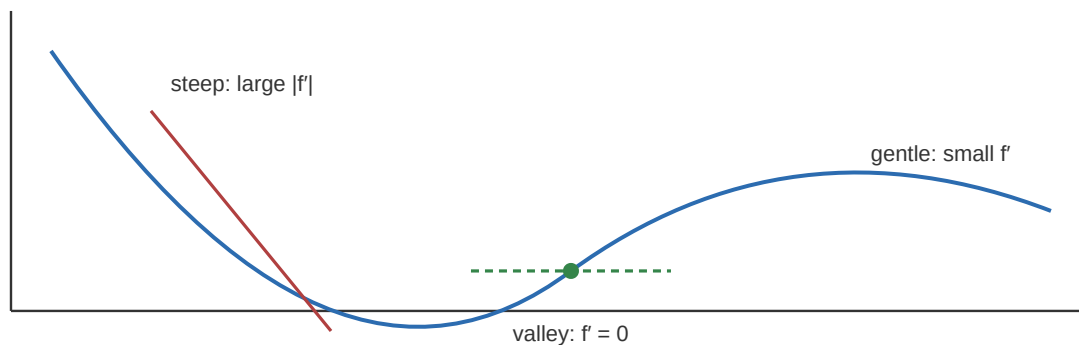
Derivatives and Gradient Descent: How Machines Learn

10.1 The derivative is an instantaneous slope

You know the slope of a line: rise over run. A curve has a different slope at every point. The **derivative** $f'(x)$ (also written $\frac{df}{dx}$) is the slope of the curve *right at* x — found by taking the rise-over-run of a tiny step and imagining the step shrinking to nothing.

$$f'(x) = (\text{steepness at } x) \approx \frac{f(x+h) - f(x)}{h} \text{ for a tiny } h$$

Everything you need to read the papers reduces to one sentence: **the derivative tells you which way, and how sharply, the output changes when you nudge the input.** Positive derivative: output rises as input rises. Zero derivative: you are at a flat spot — a peak, a valley, or a plateau. That last fact is the engine of all optimization, because *the best settings live where the derivative is zero.*



The derivative is the tangent's slope. Steep parts of the curve have large derivatives; the bottom of a valley has zero. Training a model means rolling downhill on a curve like this — and the derivative is the compass that says which way is down and how steep.

10.2 The gradient: the derivative for many knobs at once

A neural network does not have one knob; it has millions, bundled into a vector θ (Chapter 1). Its loss $L(\theta)$ is a landscape over that million-dimensional knob-space. The **gradient** ∇L ("grad L") is the vector of all the partial derivatives — one per knob, each saying "how does the loss change if I nudge *this* knob, holding the rest fixed." Its geometric meaning is the whole game:

Definition — the gradient points uphill, fastest

$\nabla L(\theta)$ is the direction in knob-space along which the loss increases most steeply, and its length is how steep that is. Therefore $-\nabla L(\theta)$ points in the direction of *fastest decrease* — the direction to move the knobs to reduce error most quickly. At a minimum, the gradient is the zero vector: no direction goes downhill.

10.3 Gradient descent: the algorithm behind every trained model in the repo

Now the single most important algorithm in the entire program, and it is three lines. To train a model, repeat: measure the gradient of the loss, then take a small step in the opposite direction.

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

Piece	Meaning
θ	all the model's knobs (weights)
$\nabla L(\theta)$	the uphill direction of the loss at the current knobs
$-\nabla L(\theta)$	therefore the downhill direction — where error falls fastest
η ("eta")	the learning rate : how big a step to take (the papers use values like 0.002, 0.0005)
$\leftarrow$	assignment: overwrite the knobs with their improved values, then repeat

Intuition pump — descending a mountain in fog

You are on a mountainside in thick fog, wanting the valley. You can't see far, but you can feel the slope under your feet. Strategy: feel which way is downhill (the gradient), step that way, feel again, repeat. Big steps (large η) descend fast but may overshoot the valley and bounce off the far wall; tiny steps are safe but slow. That is gradient descent, and that is the entire training procedure behind every "we trained an MLP" sentence in the papers. Variants you'll see named — **SGD** (feel the slope from a random handful of examples each step, for speed), **Adam** / **AdamW** (adapt the step size per knob and add momentum so you coast through flat stretches) — are all this same fog-walk with smarter footwork.

In our papers — a training loop you can now fully read

The concerned-syntax learned agents train a logistic-regression classifier by exactly this loop. The repo's code reads:

$$\begin{aligned} err &= pred - label \\ weights[j] &-= \eta \cdot (err \cdot x[j] + \lambda \cdot weights[j]) \\ bias &-= \eta \cdot err \end{aligned}$$

Decode: err is how wrong the prediction was (Chapter 2's $\hat{y} - y$). $err \cdot x[j]$ is the gradient of the loss with respect to weight j — "blame each input feature in proportion to how much it contributed to the error." The $\lambda \cdot weights[j]$ term is a gentle pull toward zero (L2 regularization, Section 10.5). Multiply by $-\eta$ and add: that is $\theta \leftarrow \theta - \eta \nabla L$, live, in a

file you can open. Once you see it here, you see it everywhere: REINFORCE, the ΔE -prediction heads, the neural weakness sweeps — all the same descent, differing only in what L measures.

10.4 The chain rule and backpropagation, in one breath

Networks are compositions (Chapter 2): loss depends on the output, which depends on the last layer, which depends on the previous layer... To get the gradient through such a stack you need the **chain rule**: the derivative of a composition is the product of the derivatives of its stages. If $y = f(g(x))$, then $y' = f'(g(x)) \cdot g'(x)$ — "multiply the sensitivities as you pass through each stage." Applied to a deep network from the loss backward toward the inputs, the chain rule is **backpropagation**: the algorithm that computes every knob's gradient in one efficient sweep. You will rarely need to compute one by hand to read the papers, but you should know that "backprop" is not magic — it is the chain rule bookkept carefully, and it is why "train the network" is a feasible instruction at all.

10.5 Reading the landscape: minima, sharpness, and regularization

Three landscape ideas recur in the weakness papers as *rival* explanations that the program tests and mostly rejects:

- **Local vs global minima.** Descent stops at whatever valley it rolls into, which need not be the deepest. Different seeds land in different valleys — one reason the papers average over seeds.
- **Flat vs sharp minima.** A minimum is "flat" if the loss barely rises as you step away (small curvature — small second derivatives) and "sharp" if it rises steeply. A popular theory holds flat minima generalize better. The weakness benchmark measures sharpness (via a clever random-probe trick called the *Hutchinson estimator*, which peeks at the curvature matrix without building it) and reports that **sharpness does not predict which rule generalizes** — weakness does. Knowing what "sharpness" means lets you appreciate that this is a real horse-race, honestly run and won by the newcomer.
- **Regularization.** Adding a penalty to the loss — most commonly $\lambda \|\theta\|^2$, punishing large weights — biases descent toward simpler solutions. The λ is a dial (Chapter 1); the compatibility-*regularization* experiments add a penalty rewarding symmetry-preservation instead, and watch OOD accuracy climb as λ rises. Every "we added an auxiliary loss with weight 0.5" is an extra term bolted onto L before descent runs.

Pitfall — the derivative that isn't there

Gradient descent needs a slope, so it needs a *smooth* loss. Several papers hinge on what happens at a *discontinuity* — a reward rule that flips sign exactly at energy $E = 0.5$. A smooth network can approximate the flip everywhere except the razor's edge, where no finite slope exists; the regime-sensitive papers show the residual failure sits precisely at that measure-zero point, and (because the agent's trajectory lingers there) it still costs real return. When you see "the failure is at the boundary," read: the thing being learned has no derivative there, so a descent-trained smooth function cannot capture it without an explicit boundary feature or a mixture-of-experts gate.

10.6 What you can now read

$f'(x)$ and ∂ as "how output moves when you nudge input"; the gradient ∇L as the uphill vector and $-\nabla L$ as downhill; $\theta \leftarrow \theta - \eta \nabla L$ as the universal training step; η as step size, SGD/Adam as its variants; the chain rule as backprop; local/global and flat/sharp minima; L_2 and auxiliary-loss regularization with weight λ ; and why discontinuities defeat descent.

CHAPTER 11

Integrals and Constrained Optimization: Budgets and Their Prices

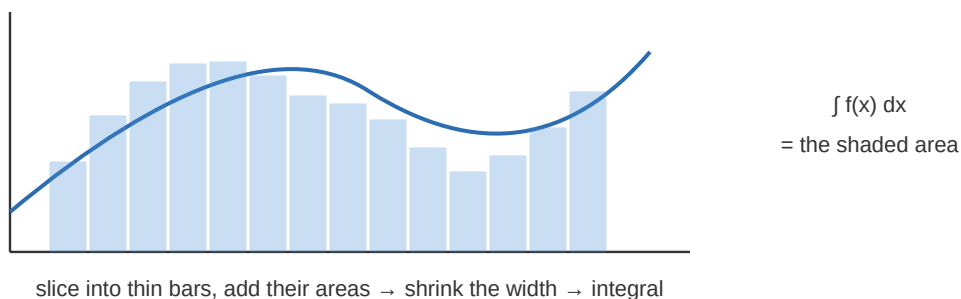
The theoretical papers — concern-weighted weakness, gauge-fixed transport, rate-distortion — optimize something subject to a *budget*: allocate limited resolution, spend a fixed capacity. Two tools handle this: the **integral** (adding up a varying quantity over a whole space) and the **Lagrange multiplier** (the exchange rate between a goal and a constraint). Both are less scary than their notation.

11.1 The integral is a sum with infinitely many, infinitely thin pieces

Σ (Chapter 1) adds up a finite list. When the thing you're adding varies *continuously* — concern spread over a space, resolution over a region — you slice the region into infinitely many infinitely thin pieces and add those. That limit is the **integral**:

$$\int f(x) dx = (\text{the total of } f, \text{ accumulated across all } x) = \text{area under the curve } f$$

The elongated S is just "sum"; dx is the width of an infinitely thin slice. Read $\int f(x) dx$ as "add up f over everything." That is all you need. When the concern-weighted-weakness paper writes the continuous version of concern mass as $W_c(h) = \int_{Z_h} d\kappa(x)$, it means exactly what its discrete cousin $\Sigma \kappa(s)$ meant — "total up the concern over the compatible region" — with a smooth space instead of a countable list.



An integral is the area under a curve. Approximate it with thin rectangles, then let their width shrink to zero. Where a sum totals a spreadsheet column, an integral totals a continuous quantity — concern over a region, error over a space, probability over an interval.

11.2 Constrained optimization: the best you can do on a budget

Unconstrained optimization (Chapter 10) rolls downhill freely. But real problems have a leash: "minimize error *subject to* using no more than capacity B ." The rate-distortion setup is exactly this —

$$\text{minimize}_p \int w(x) D(p(x)) dx \quad \text{subject to} \quad \int p(x) dx \leq B$$

Read it: "choose a resolution plan ρ that minimizes value-weighted distortion D , without spending more than budget B of total resolution." The left integral is the cost you want small; the right integral is the resource you can't exceed. This is the mathematical form of "spend limited attention where it matters most," and it appears in nearly every theoretical result in the program.

11.3 The Lagrange multiplier is a price

Definition — Lagrange multiplier

To optimize with a budget constraint, invent a single number λ — the **Lagrange multiplier** — that acts as the *price* of the resource, and instead optimize (goal) $-\lambda \cdot (\text{resource used})$, freely. At the best allocation, λ is exactly the *exchange rate*: how much your goal would improve if the budget grew by one unit ("the marginal value of one more unit of capacity"). The constraint disappears into a price, and ordinary calculus (set the derivative to zero) finishes the job.

Intuition pump — the shadow price of a rule you can't break

A factory maxing out profit under a hard limit of 100 machine-hours will, at the optimum, have a number in mind: "one more machine-hour is worth \$40 to me." That \$40 is the Lagrange multiplier — the *shadow price* of the constraint. Nobody wrote it on the wall; it emerges from the optimization. Lagrange's insight is that you can run the whole problem *forward* from this price: pretend hours cost \$40 each, optimize with no limit, and you land exactly on the constrained optimum. Every constraint has such a hidden price, and λ names it.

11.4 Worked derivation: where the reward-deformation exponent comes from

You now have every tool to follow the program's signature derivation end to end — the one that predicts $\rho^*(x) \propto w(x)^{d/(d+2)}$. Watch how few moves it takes.

Visual proof — the power law falls out of one derivative

1. **Distortion vs resolution.** A basic coding fact: if you spend resolution density ρ at a point in a d -dimensional space, the leftover error there scales as $D \propto \rho^{-2/d}$ — more resolution, less error, with a dimension-dependent exponent. (More pixels $\rightarrow$ sharper image, and how fast depends on how many dimensions you're tiling.)
2. **Set up the priced objective.** Fold the budget into a price λ :

$$L[\rho] = \int [w(x) \cdot \rho(x)^{-2/d} - \lambda \cdot \rho(x)] dx$$

minimize value-weighted error minus the price of the resolution you buy.

3. **Set the derivative to zero.** Differentiate the bracket with respect to ρ at each point (power rule from Chapter 10: the derivative of $\rho^{-2/d}$ is $(-2/d)\rho^{-2/d-1}$) and set it to 0:

$$-\frac{2}{d} w(x) \rho(x)^{-(d+2)/d} - \lambda = 0$$

4. **Solve for ρ .** Rearranging (pure algebra — move terms, raise both sides to a power) gives

$$\rho^*(x) \propto w(x)^{d/(d+2)}$$

the reward-deformation law. Plug $d = 2$ to get the predicted exponent 0.5.

Four moves, one prediction. A coding fact, a priced objective, a zero-derivative condition, and algebra. No step is beyond this book — and the output is a falsifiable exponent that a log–log slope later measured at ≈ 0.30 , revealing an effective dimension near 1. This is the program in miniature: derive a sharp number before measuring, then let the measurement talk back.

11.5 What you can now read

$\int f \, dx$ as "total up f over everything" / area under a curve; the continuous concern-mass $\int d\kappa$; constrained problems "minimize ... subject to ... $\leq B$ "; the Lagrange multiplier λ as the shadow price of a budget; the zero-derivative optimality condition; and the full derivation of $\rho^* \propto w^{d/(d+2)}$.

CHAPTER 12

Information Theory: Surprise, Bits, and the Cost of Being Wrong

Information theory quantifies surprise. It gives the program its notion of "bits," its description-length rival to weakness, the cross-entropy loss that trains its classifiers, the KL divergence that measures how far one distribution sits from another, and the rate–distortion trade that anchors the geometry results. All of it grows from one definition.

12.1 Surprise = log of one-over-probability

How surprising is an event of probability p ? Shannon's answer: rarer should mean more surprising, and independent surprises should add. The only measure that does both is

$$\text{surprise}(p) = \log_2 \frac{1}{p} = -\log_2 p \text{ bits}$$

A certain event ($p = 1$) carries 0 bits of surprise; a coin flip ($p = \frac{1}{2}$) carries 1 bit; a 1-in-1000 event carries about 10 bits. This is the same log-of-a-count from Chapter 3, now read as "how many yes/no questions of news did this event deliver." The minus sign is only there because probabilities are ≤ 1 , so their logs are negative and we want a positive amount of surprise.

12.2 Entropy is average surprise

Definition — Shannon entropy

$$H(p) = \sum_{\text{outcomes}} p \cdot \text{surprise}(p) = -\sum p \log_2 p$$

Entropy is the expected surprise of a distribution — its average unpredictability, in bits. A fair coin has $H = 1$ bit; a two-headed coin has $H = 0$ (no surprise, ever); a fair 8-way die has $H = 3$ bits. Entropy is maximized by uniformity (all outcomes equally likely) and minimized by certainty. It is the exact number of bits you'd need, on average, to record outcomes from the source.

Entropy shows up in the program as the quiet engine under several named quantities. The **effective rank** / **participation ratio** of Chapter 8 is entropy applied to a spectrum: treat the normalized eigenvalues as a probability distribution and take $\exp(H)$ — you get "how many directions are effectively in play." The concerned-syntax agents measure an intervention's **information gain** as $\log_2$ of the number of distinct outcomes it can produce — 1 bit if a probe distinguishes two candidate parses, 0 if it can't tell them apart. Same log, same bits, applied to "how much did I learn."

12.3 Cross-entropy: the price of a wrong codebook, and the classifier's loss

Suppose reality follows distribution p but you plan (encode, predict) as if it followed q . Your average surprise is now the **cross-entropy** $H(p, q) = -\sum p \log q$ — always at least the true entropy $H(p)$, with the excess being the toll for believing the wrong distribution.

In our papers — cross-entropy is the training signal

Every classifier in the repo — the sensory and valence encoders, the parse predictors, the action heads — is trained by minimizing cross-entropy between the true label and the model's predicted probabilities. In practice the true label is a one-hot vector (Chapter 7), so $-\sum p \log q$ collapses to $-\log q(\text{correct class})$: "pay the surprise you assigned to the right answer." The model minimizes this by putting more probability on correct answers — descent (Chapter 10) on cross-entropy is how a network is taught to classify. The "cross-entropy sign loss" in the regime-sensitive papers is this same idea applied to getting the *sign* of an action-advantage right. And the commitment-surface papers measure causal use as a **patch cross-entropy increase**: how much worse the model's predictions get (how much extra surprise it eats) when you surgically corrupt the feature under test. A feature that's truly load-bearing, when patched, spikes the cross-entropy; a decorative one doesn't.

12.4 KL divergence: how far apart two distributions are

Definition — Kullback–Leibler divergence

$$KL(p \parallel q) = H(p, q) - H(p) = \sum p \log \frac{p}{q} \geq 0$$

The **KL divergence** is the *extra* bits paid for using q when the truth is p — the pure penalty, with the unavoidable entropy subtracted off. It is zero exactly when $p = q$ and grows as they diverge. It is not symmetric ($KL(p \parallel q) \neq KL(q \parallel p)$), so it is a "divergence," not a distance — a directed measure of "how surprised will I be, in p 's world, if I packed for q 's."

KL is the natural yardstick whenever the papers compare a learned distribution to a target, and it is one of several distances the gauge-transport paper lists as admissible ways to measure how much a distinction degrades under transport (alongside total-variation and Wasserstein distance — other principled ways to score "how different are these two distributions," each sensitive to different kinds of difference). You do not need their formulas to read the papers; you need to know that all three answer one question — *how far apart are these two probability distributions?* — and that KL answers it in the currency of bits.

12.5 Rate–distortion and MDL: two faces of "compress what matters"

Two program-critical ideas are information theory wearing work clothes.

- **Rate–distortion theory** asks: with a limited bit budget (the *rate*), how small can I make my reconstruction error (the *distortion*)? It is the information-theoretic form of the budget problem in Chapter 11, and it is where the $D \propto \rho^{-2/d}$ fact comes from. "Spend bits where value is high" is rate–distortion with a concern weight — the entire metric-deformation story, restated: the code is a lossy compressor that allocates its rate by concern.

- **Minimum Description Length (MDL)** says the best hypothesis is the one that lets you *compress the data most* — shortest (model + data-given-model) in bits. It is Occam's razor made quantitative, and via Chapter 3's 2^{-L} it equals a preference for a-priori-probable short programs. MDL is a *rival* to weakness, and the program's flagship result is a head-to-head: on tasks engineered so a short shortcut and a weak symmetric rule both fit the training data, **MDL / compression / description-length selectors pick the shortcut and generalize badly, while the weakness selector picks the symmetric rule and generalizes**. Understanding MDL as "shortest code wins" is what makes that horse-race legible — and makes the newcomer's victory meaningful rather than definitional.

Intuition pump — short is not the same as weak

"The password is 0000" is a very *short* hypothesis. "The password is some 4-digit number" is a very *weak* one (it allows 10,000 possibilities). Short commits hard to one guess; weak stays broadly compatible with many. The program's wager is that when you don't yet know which world you're deploying into, the weak-but-compatible rule travels better than the short-but-brittle one — because it hasn't bet the farm on the training distribution's quirks. Chapters 3 and 4 gave you the logs and counts; Chapter 19 makes the wager precise; here the point is just that *description-length short* and *extension-weak* are genuinely different rulers, and the papers measure both to prove it.

12.6 What you can now read

Surprise = $-\log_2 p$; entropy $H = -\sum p \log p$ as average surprise / bits; entropy behind effective rank and information gain; cross-entropy as the classifier's loss and as patch-CE causal-use measure; KL divergence as extra bits / directed distance between distributions (with TV and Wasserstein as cousins); rate-distortion as the bit-budget trade behind metric deformation; and MDL as "shortest code," the rival that weakness beats.

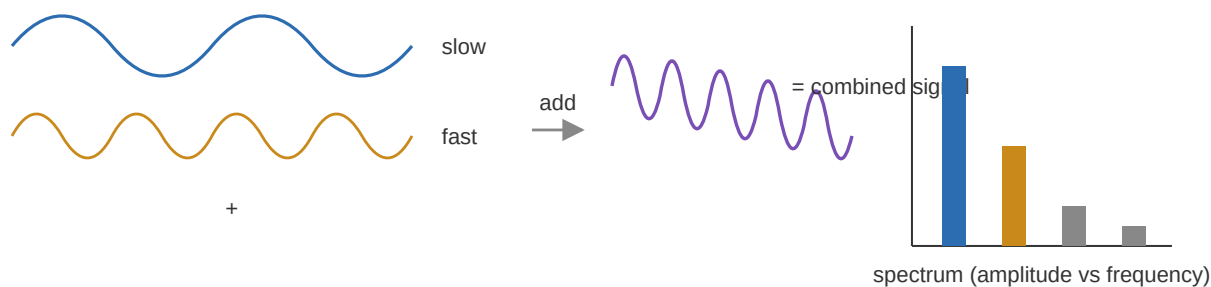
CHAPTER 13

Waves and Spectra: Fourier Thinking

Your trig class taught sine and cosine as triangle ratios. This chapter reveals their second career: sines and cosines are the *atoms of every repeating pattern*, and decomposing a signal into them — Fourier analysis — is how the program's grid-cell, spectral-bias, and feature-encoding results work. It is also the hidden link between "symmetry," "frequency," and "grid," so a little of it pays off widely.

13.1 Any pattern is a sum of waves

A sine wave $\sin(kx)$ wiggles k times as fast as $\sin(x)$; k is its **frequency**. Fourier's astonishing theorem: *any* reasonable repeating function — a square wave, a spike train, a birdsong, a neuron's firing map — can be built by adding sine and cosine waves of different frequencies, each with its own amplitude. The list of those amplitudes is the signal's **spectrum**: a recipe that says "this much of the slow wave, this much of the fast one." Two representations of the same object — the wiggle in time, and the recipe in frequency — related by the **Fourier transform** (its discrete, computational form is the DFT).



Time-view and frequency-view of one signal. Left: two pure waves add into a complicated shape. Right: the spectrum stores the same information as a few bars — how much of each frequency is present. A sharp, spiky signal needs many high-frequency bars; a smooth one is dominated by low frequencies. The whole of "spectral bias" and "band-limiting" is bookkeeping on these bars.

13.2 Fourier features: why the agency papers feed networks sine waves

Recall the recurring input list $[E, \sin(\pi E), \cos(\pi E), \sin(2\pi E), \cos(2\pi E), \sin(4\pi E), \cos(4\pi E)]$. Now it reads plainly: alongside the raw energy E , the network gets E encoded at several frequencies. Why bother? Because plain networks have a **spectral bias** — left alone, gradient descent learns smooth, low-frequency functions first and struggles to represent sharp changes (like the reward flip at $E = 0.5$). Handing the network pre-made high-frequency waves lets it assemble a sharp function by mixing them, the way the spectrum above builds a spiky shape from a few bars. These **Fourier features** are a standard trick, and the regime-sensitive papers test exactly whether they help a smooth network capture a boundary — a question you can now see is really "can a few extra frequency bars approximate a step?"

13.3 The deep link: frequency is symmetry (grid cells)

Here is where Fourier thinking becomes central rather than convenient — the grid-cell weakness result. When a system is symmetric under shifts (translations), the Fourier waves are special: shifting the input only *rotates the phase* of each wave, never mixes frequencies. In the language of Chapter 14, the waves $\chi_k(x) = e^{2\pi i k x/n}$ are the **irreducible representations** ("irreps") of the cyclic shift group — the elementary, non-mixing patterns that symmetry permits. So a code that respects translation symmetry will naturally organize itself into a few frequency components. That is precisely what **grid cells** are: neurons whose firing is a periodic (roughly sinusoidal) function of position — biological Fourier modes. "Grid cell of spatial period n/k ," "Fourier mode χ_k ," and "irrep k of the shift group" are, the synthesis note argues, three names for one thing.

In our papers — one event, three instruments

The grid-cell weakness program predicts that when a path-integration network discovers the translation group, three measurements should all light up together, because they are measuring the same discovery: (1) **weakness** — the code stays consistent under many shifts (Chapter 19); (2) **spectrum** — the code concentrates into a few Fourier modes, measured by the participation ratio of the DFT (a low participation ratio = few active frequency bars); (3) **topology** — those few periodic modes trace out a torus (Chapter 15). The "participation ratio" is entropy (Chapter 12) on the spectrum: how many frequency bars are meaningfully lit. When a paper says "toroidal topology and spectral concentration and translation weakness co-occur," it is claiming these three needles swing together — and the experiment is designed to check whether they do.

13.4 What you can now read

Frequency k ; any pattern as a sum of sine/cosine waves; the spectrum as amplitude-vs-frequency and the DFT as its computation; Fourier features as multi-frequency inputs that defeat spectral bias; spectral bias itself (smooth-first learning); participation ratio as entropy on a spectrum; and the grid-cell equivalence "frequency mode = irrep of the shift symmetry = periodic grid code," the bridge into the next two chapters.

PART IV

Symmetry and Shape

Two branches of modern math describe structure that survives change: **group theory** (the mathematics of symmetry) and **topology** (the mathematics of shape up to bending). Both are central to the program — weakness is defined in terms of symmetry, and the grid-cell results turn on topology — yet neither is hard once you see what it is really about. This part builds both from everyday intuition.

CHAPTER 14

Groups, Symmetry, and Equivariance

14.1 A symmetry is a change that changes nothing

Rotate a square by 90° and it looks identical. Flip it across a diagonal — identical again. A **symmetry** of an object is a transformation that leaves it looking the same. The striking fact is that an object's symmetries always fit together with beautiful regularity: do two in a row and you get a third; every symmetry can be undone; doing nothing counts. Any collection of transformations closed under these rules is a **group**.

Definition — group

A **group** G is a set of transformations with: (1) *closure* — combining two gives another in G ; (2) an *identity* e — the "do nothing" transformation; (3) *inverses* — every g has an undo g^{-1} ; (4) *associativity* — grouping doesn't matter. That is the whole definition. "Group" is just the precise word for "a self-consistent world of symmetries."

The groups the papers use are small and concrete:

Group	What it is	Where it appears
$\mathbb{Z}_n$ (cyclic)	the n rotations of a ring; "add $k \bmod n$ "	the shift tasks: $f(x) = (x + b) \bmod n$; grid-cell translations
D_n (dihedral)	rotations <i>and</i> reflections of an n -gon	the reflection tasks: $f(x) = (b - x) \bmod n$
S_n (symmetric)	all $n!$ reorderings of n items	permutation controls; a "hard" symmetry family
$\mathbb{Z}_2$ (parity)	two elements: identity and one flip	parity/XOR tasks; the simplest nontrivial symmetry

"mod n " (modular arithmetic) is clock arithmetic: on a 12-hour clock, $10 + 5 = 3$, because you wrap around. $\mathbb{Z}_n$ is exactly the clock with n hours, and "shift by b " is moving every hour-hand forward by b . When a paper writes the ground-truth rule as $(x + b) \bmod n$, it is describing a rotation of an n -point ring — a symmetry the right hypothesis should respect.

14.2 Group actions and orbits

A group *acts* on a set when each group element rearranges the set's members — $g \cdot x$ means "apply symmetry g to thing x ." Starting from one x and applying every group element sweeps out its **orbit**: the full set of things reachable from x by symmetry. On a ring under rotations, one point's orbit is the whole ring; under a smaller subgroup, just part of it. Orbits are "everything that is the same as x , according to this symmetry" — the program's operational notion of a category, and the reason "same-orbit" appears in the cluster-gap metric (Chapter 7).

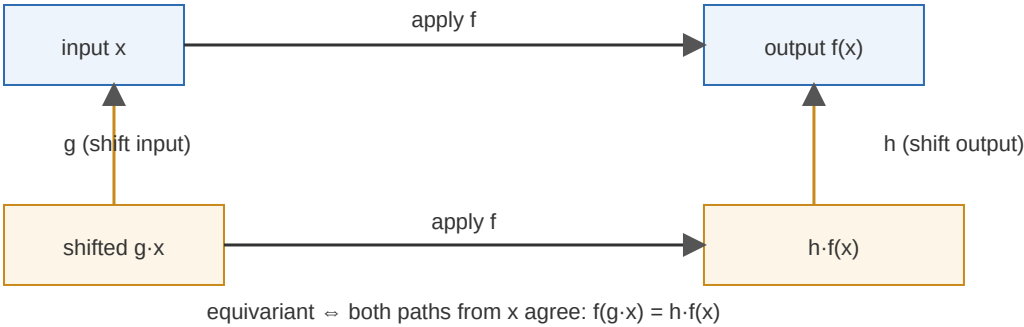
14.3 Equivariance and invariance: the crux

Now the concept the whole weakness arc is built on. A function can relate to a symmetry in two ways:

Definitions — invariance and equivariance

Invariant: transforming the input doesn't change the output at all — $f(g \cdot x) = f(x)$. ("Rotate the digit, the classifier still says 7.")

Equivariant: transforming the input transforms the output *correspondingly* — $f(g \cdot x) = h \cdot f(x)$ for some matching output-transform h . ("Shift the input by 3, the output shifts by 3 too.") Invariance is the special case where h does nothing.



Equivariance is a commuting square. Start at "input x ." You can go right then down (apply f , then shift the output) or down then right (shift the input, then apply f). The function is equivariant with respect to the symmetry exactly when both routes land on the same value. "Compatible with the symmetry" means this square commutes.

14.4 From equivariance to weakness

Everything converges here. Count *how many* of a group's symmetries a function is compatible with, and you have the program's central quantity. This is the exact definition from the flagship paper (now fully readable):

$$W_G(f) = | \{ g \in G : \exists h \in G, \forall x, f(g \cdot x) = h \cdot f(x) \} |$$

Piece	Read as (using this book)
$\{ g \in G : \dots \}$	set-builder filter (Ch 1): the symmetries g that pass a test
$\exists h \in G, \forall x$	"there exists an output-transform h such that, for every input x ..." (Ch 1 quantifiers)
$f(g \cdot x) = h \cdot f(x)$	...the equivariance square commutes (this section)
$ \cdot $	count them (Ch 4)

In English: **weakness is the number of the group's symmetries that the function respects.** A function compatible with many symmetries is barely committed to specifics — it treats whole orbits alike — hence "weak" in the sense of ruling out few possibilities. Chapter 19 develops why that predicts generalization; for now, notice that you just read, unaided, the equation that anchors a dozen papers.

14.5 Representations and irreps: symmetry as matrices

To let symmetry act on vectors (neural activations), each group element is realized as a matrix $\rho(g)$ — a **representation**. Then equivariance reads $f(g \cdot x) = \rho(g) f(x)$: the layer commutes with the symmetry's matrices. A representation can often be broken into indivisible building-block patterns — the **irreducible representations** ("irreps"). For the cyclic shift group, the irreps are exactly the Fourier waves $e^{2\pi i k x/n}$ of Chapter 13. That is the rigorous version of "frequency = symmetry": the elementary patterns compatible with translation symmetry *are* the frequency modes, which is why a translation-respecting neural code spontaneously organizes into grid-like periodic cells. Group theory and Fourier analysis are two dialects for one structure.

Intuition pump — why symmetry is "the portable language of constraints"

The program's thesis sentence is that geometry (and symmetry) is how adaptive systems encode constraints portably. Here's why symmetry earns that role. A constraint says "these situations must be treated alike." A symmetry group is *exactly* a bookkeeping system for "alike": its orbits are the equivalence classes, its elements are the allowed ways to move within them. Any system that must generalize — treat unseen-but-equivalent cases correctly — is implicitly betting on a symmetry. Measuring weakness measures how big and how consistent that bet is. That is why one number, computed the same way, keeps predicting generalization across vision, language, and navigation: they are all making symmetry bets, and weakness reads the size of the bet.

14.6 What you can now read

Group axioms (closure, identity, inverse); $\mathbb{Z}_n/D_n/S_n/\mathbb{Z}_2$ and modular "clock" arithmetic; group actions $g \cdot x$ and orbits as categories; invariance $f(g \cdot x) = f(x)$ vs equivariance $f(g \cdot x) = h \cdot f(x)$ as a commuting square; the full weakness definition $W_G(f)$; representations $\rho(g)$ and irreps as Fourier modes; and why symmetry is the natural language of constraints and generalization.

CHAPTER 15

Topology: Holes, Tori, and the Shape of a Code

Topology studies the properties of shape that survive stretching and bending — what stays true if you make an object out of rubber and deform it without tearing. It sounds abstract, but the program uses it for something concrete and beautiful: to detect that a neural population code has wrapped itself onto a *torus* (a donut), which is the fingerprint of a system that has discovered 2-D spatial symmetry. This chapter gives you exactly the topology the grid-cell papers need.

15.1 The topologist's question: how many holes?

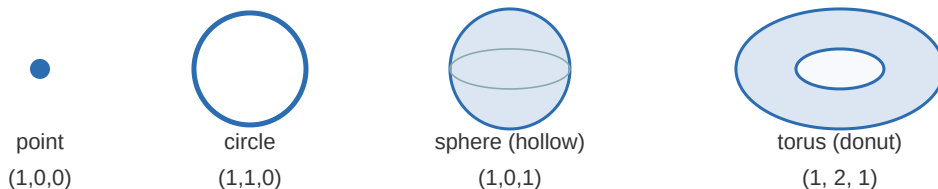
To a topologist a coffee mug and a donut are the same object — each has exactly one hole (the handle, the donut's center), and you could mold one into the other without tearing. Distance and angle don't matter; *connectivity* does. The coarsest, most robust description of a shape is its inventory of holes, and holes come in dimensions.

Definition — Betti numbers

The **Betti numbers** ($b_0, b_1, b_2, \dots$) count holes by dimension:

- b_0 = number of separate *connected pieces*.
- b_1 = number of independent *loops* (1-D holes you could lasso — a circle has one, a figure-eight two).
- b_2 = number of enclosed *voids / cavities* (2-D holes, like the hollow inside a balloon).

They are the shape's robust signature: bend and stretch all you like, the hole-counts don't change.



Shapes by their hole-inventory (b_0, b_1, b_2). A point: one piece, no holes. A circle: one piece, one loop. A hollow sphere: one piece, no loops, one enclosed void. A torus: one piece, two independent loops (around the tube and around the hole), one void — the signature (1, 2, 1). That last triple is the one the grid-cell papers hunt for.

15.2 Why a torus means "discovered 2-D space"

Recall from Chapter 13 that a translation-respecting code organizes into periodic (circular) Fourier modes. Now stack two independent spatial directions — say, moving east and moving north. Each is periodic, so each traces a *circle* in the code's activity. Two independent circles, combined, make a **torus** — because a torus is by definition the product of two circles (one loop going around the tube, one going around the hole). So if a navigating network has truly

discovered 2-D translational structure, its population activity should live on a torus, and that shows up as Betti numbers (1, 2, 1). Measuring the topology of the activity is therefore a way to *read off whether the system found the symmetry* — independent of, and convergent with, the weakness and spectral measurements of Chapter 13.

Intuition pump — a torus is a screen that wraps both ways

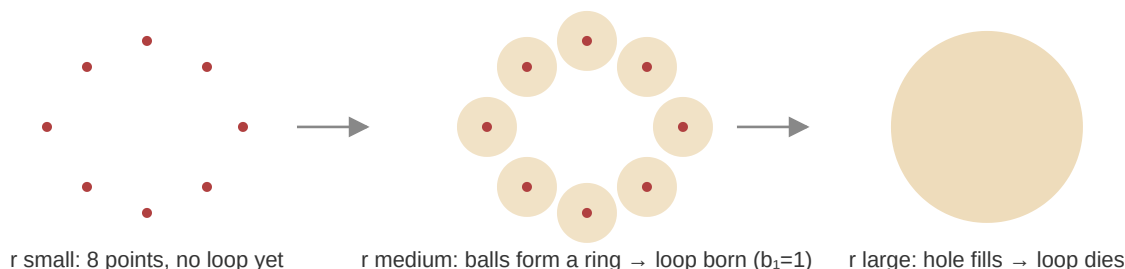
Picture the old arcade game where flying off the right edge brings you back on the left, and off the top brings you back at the bottom. That wrap-around screen is a torus: horizontally it's a circle (right wraps to left), vertically another circle (top wraps to bottom). A grid cell fires periodically as you move — cross one spatial period and you're "back where you started" in the code — so the animal's whole 2-D position maps onto this wrap-around screen. Finding a torus in neural data is discovering that the brain built the arcade screen. The two loops ($b_1 = 2$) are the two wrap directions; the enclosed void ($b_2 = 1$) is the donut's hollow. It is a very specific fingerprint, which is what makes it strong evidence.

15.3 Persistent homology: measuring holes in noisy point clouds

Real neural data isn't a clean donut; it's a fuzzy cloud of points. How do you count holes in a cloud? **Persistent homology** answers this with a clever moving picture. Put a ball of radius r around every data point and grow r from zero:

- At $r = 0$, you have isolated points ($b_0 = \text{number of points}$).
- As r grows, nearby balls merge (pieces join, b_0 falls) and loops appear as balls encircle a gap, then fill in as r grows further.
- A hole that *persists* across a wide range of r is a real feature of the shape; one that appears and vanishes immediately is noise.

The output is a list of features with their birth-and-death radii; long-lived ones are the true Betti numbers. This is how the grid-cell experiments compute a "toroidal score" — a persistence-weighted measure of how close the population cloud's holes are to the torus signature (1, 2, 1). "Persistent" is the key word: it is a built-in noise filter, the topological cousin of the "CI-lower-bound-above-zero" gates from Chapter 6 — only durable features count.



Persistent homology grows balls and watches holes. A loop that is born early and dies late (persists across many radii) is a genuine hole; brief flickers are noise. The span from birth to death is the feature's "persistence." Software (ripser, gudhi) computes these automatically; the papers turn the result into a single toroidal score.

In our papers — topology as one of three convergent needles

The grid-cell weakness preregistration ties topology to the program's core claim with a *mediation* test (Chapter 6): it predicts that translation weakness will correlate with out-of-distribution generalization, and that this correlation runs *through* the toroidal topology — formally, that the partial correlation of weakness with OOD, controlling for the toroidal score, should drop by at least half. If it does, topology isn't a bystander; it is the mechanism by which weakness buys generalization. This is the same residualizing move from Chapter 6, now with a Betti-number-derived score as the thing controlled for — a clean example of how the program fuses topology, symmetry, and statistics into a single falsifiable prediction.

15.4 What you can now read

Topology as shape-up-to-bending; Betti numbers (b_0 , b_1 , b_2) as counts of pieces, loops, and voids; the torus as the product of two circles with signature (1, 2, 1) and its meaning as "discovered 2-D symmetry"; persistent homology as growing balls to count durable holes in noisy clouds; the toroidal score; and topology's role as a mediating variable linking weakness to generalization.

PART V

Agency, Dynamics, and Causality

The second half of the research program is about *agency* — systems that keep themselves alive, that act, that care. Its mathematics is dynamical systems (how things evolve over time), decision theory (how to act well), and causality (how to tell what really drives what). These three chapters give you all of it, again from the ground up, and connect each tool to the specific agency papers that use it.

CHAPTER 16

Dynamical Systems, Control, and Attractors

16.1 A dynamical system is a rule for the next moment

A **dynamical system** is nothing but a state plus an update rule: "given where you are now, here is where you'll be next." The agency papers' entire world is one such rule, and you already met it in Chapter 2. An agent has an internal energy E , and each time step:

$$E \leftarrow \text{clip}(E - 0.04 + \text{reward}, 0, 1)$$

Each step, energy decays by 0.04; eating adds reward; the result is pinned to [0,1]; the episode ends if E hits 0. That's the physics of being alive in these papers.

Iterate an update rule and you get a **trajectory** — the sequence of states over time, a path through the state space of Chapter 2. The two-variable version adds damage D ($D \leftarrow D + 0.03$, terminate if $D \geq 1$), so the state is a point in the unit square moving under the rule, and the agent's job is to steer that point away from the death boundaries. Everything the control papers study is the geometry of these trajectories.

16.2 Feedback control: setpoints, error, and correction

A **control loop** keeps a quantity near a desired **setpoint** by measuring the **error** (setpoint – current) and acting to shrink it. A thermostat is the canonical example: too cold $\rightarrow$ heat, too hot $\rightarrow$ cool. This is the mathematics of **homeostasis** (staying alive by keeping vital signs in range) and its more forward-looking cousin **allostasis** (acting *in advance* to stay in range).

Intuition pump — the governor that gave "cybernetics" its name

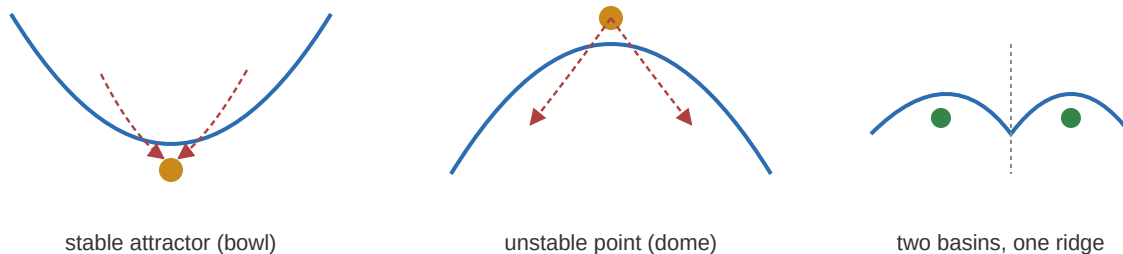
A steam engine's spinning governor: as it speeds up, weights fly outward and pinch the throttle, slowing it; as it slows, they drop and open the throttle. No intelligence, just a loop that converts "too fast" into "less fuel." The word *cybernetics* comes from the Greek for steersman. The program's "virtual governor" papers ask a modern version: can a global constraint (system-level stress) be transduced into a *local* signal that individual parts can act on, so that each part minimizing its own local discomfort also reduces global stress? The math is feedback control; the twist is making the feedback signal itself a designed, learnable interface — and the controls (stale signal, wrong signal, no signal) test whether the loop's *fidelity*, not merely its presence, is what does the work.

16.3 Fixed points and attractors: where trajectories settle

A **fixed point** of an update rule is a state that maps to itself — put it in, get it back; the system rests there. Fixed points come in flavors, and the distinction is the heart of the "passive vs active geometry" question:

Definition — stable attractor vs unstable point

A fixed point is a **stable attractor** if nearby trajectories are drawn *toward* it (a marble in a bowl: nudge it, it rolls back). It is **unstable** if nearby trajectories flee (a marble balanced on a dome: the slightest nudge and it's gone). The set of starting points that eventually funnel into a given attractor is its **basin of attraction**.



The landscape metaphor. Attractors are valleys that pull trajectories in; unstable points are ridges that repel. A system's long-run behavior is decided by which basin it starts in. The program's deepest question — when does a passive representation become an active, self-maintaining regime? — is asking when a mere *location* in state space becomes a *basin* that actively restores itself after perturbation.

16.4 Self-maintenance: autopoiesis and ultrastability, made testable

This is where dynamics meets the program's boldest agency claims. A system is **autopoietic** (self-producing) if it actively maintains the very organization that keeps it a system — if, perturbed, it repairs itself back to viability. Ashby's **ultrastability** is the concrete mechanism: when a system is knocked outside its viable zone, it changes its own internal parameters until it's back inside. The autopoietic-control paper makes this measurable: it perturbs a trained model with weight noise, then lets it run a few test-time gradient steps (Chapter 10), and checks whether it climbs back to accuracy. Recovery — "repair" — is the fingerprint of an attractor that isn't just sitting there but actively defending its basin.

In our papers — the viability buffer and the Law of the Stack

Two quantities make these ideas numeric. The **viability buffer** $|B_\pi| = w(\pi) - |\alpha|$ is the slack between how many futures a policy keeps open (its weakness w — Chapter 19) and how many outcomes it has actually committed to ($|\alpha|$): buffer is room to absorb shocks without dying. The **Law of the Stack**, $w(\zeta_{i+1}) \leq 2^{w(\zeta_i)}$, bounds how much viable flexibility one layer can supply the layer above it (Chapter 3 decoded this: each bit of slack below doubles the room above). The autopoietic paper's headline test is whether a model's capacity to *repair* itself grows with the slack the Law predicts — turning a philosophical claim about self-maintenance into an ordered prediction you can check across conditions. It is a striking example of the program's method: take a concept from theoretical biology (autopoiesis), find the one inequality that makes it measurable, and pre-register a gate on it.

16.5 What you can now read

Dynamical systems as state + update rule; the homeostat $E \leftarrow \text{clip}(E - \text{decay} + \text{reward}, 0, 1)$ and its trajectories; feedback control, setpoints, error, homeostasis/allostasis; the virtual governor as a transduced global-to-local signal; fixed points, stable attractors, unstable points, and basins of attraction; autopoiesis and Ashby ultrastability as self-repair; and the viability buffer $|B_\pi| = w(\pi) - |\alpha|$ with the Law of the Stack.

CHAPTER 17

Decisions, Value, and Reinforcement Learning

An agent must choose actions. The mathematics of choosing well — decision theory and reinforcement learning (RL) — runs through the planning, concern, and benchmark papers. It is built from pieces you already have: expected value (Chapter 5), argmax (Chapter 1), and gradient descent (Chapter 10). This chapter assembles them into the agent's decision machinery.

17.1 The Markov decision process: the frame for all of it

Definition — Markov decision process (MDP)

An **MDP** is the standard model of an agent-in-a-world: a set of *states* s , a set of *actions* a , a *transition rule* giving the next state s' (possibly random), and a *reward* r for each step. "Markov" means the next state depends only on the current state and action, not the full history — the present is a sufficient summary. The agent's behavior is a **policy** π : a rule mapping states to actions (or to probabilities over actions).

The agency papers are all small MDPs: the state is (energy, damage, what item is present), the actions are {skip, consume, null, regulate}, transitions follow the homeostat rule of Chapter 16, and reward encodes what keeps the agent alive. Reading any of these papers starts with identifying these four pieces — they are always stated in the first page of the setup.

17.2 Value, discounting, and the point of γ

An action's worth is not just its immediate reward but everything it leads to. The **value** of a state (under a policy) is the total future reward expected from there. Because a reward now is worth more than the same reward later (and to keep infinite sums finite), future rewards are **discounted** by a factor $\gamma \in [0, 1)$ per step:

$$V(s) = \mathbb{E}[r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \dots]$$

Piece	Meaning
$\mathbb{E}[\cdot]$	expected value (Ch 5) — average over the randomness of the world and policy
r_t	the reward t steps from now
γ^t	discount: with $\gamma = 0.99$, a reward 100 steps out is worth $\sim 0.37\times$ its face value; with $\gamma = 0.5$, $\sim 0.001\times$

γ is a patience dial. Near 1 (the papers often use 0.99): far-sighted, the future matters almost as much as now. Near 0: myopic, only immediate reward counts. The exponent γ^t is Chapter 3's exponential decay, applied to how much the future counts. This one number quietly sets an agent's whole temporal character.

17.3 Acting greedily and planning one step ahead

Given a way to score actions, the simplest policy is **greedy**: take the best-scoring action, $a^* = \operatorname{argmax}_a \operatorname{score}(a)$ (argmax from Chapter 1 — "the winner"). The planning-from-concern paper's central, striking result is that greedy action on a learned *consequence model* is enough:

$$a^* = \operatorname{argmax}_a \Delta E_{\text{head}}(z, E, a)$$

The agent has learned a little function predicting how each action will change its energy, and it simply picks the action predicted to help most. No policy-gradient training, no long rollouts — the prediction model *becomes* the policy. The paper elevates this to a named principle, "predictive policy closure": *when an agent learns an action-conditioned model of what it cares about, that model can serve directly as the policy*. It is a clean instance of the program's taste for the simplest architecture that suffices — and it matches the performance of far more elaborate schemes.

17.4 Reinforcement learning: improving a policy from reward alone

When you don't have a consequence model, you learn the policy directly from experienced reward. The workhorse in these papers is **REINFORCE** (a policy-gradient method): try actions, see returns, and nudge the policy to make high-return actions more likely. It is gradient descent (Chapter 10) where the "loss" is engineered so that descending it raises expected return.

Intuition pump — training a dog vs reading the manual

Two ways to get good behavior. *Model-based* (greedy planning): read the manual — learn how the world responds, then reason out the best move. *Model-free* (REINFORCE): train the dog — try things, reward what works, repeat until good habits form, never understanding the mechanism. The two-bottlenecks and concern papers pit these against each other and find something subtle: a model-free agent can succeed behaviorally while its *internal representation* stays disorganized, and a well-organized representation can still fail to produce competent behavior under weak reward signal. "Behavioral success" and "understood the world" are *two* axes, not one — which is the entire thesis of the causally-grounded-agents benchmark (Chapter 18).

Two technical notes that demystify the RL vocabulary in the papers. A **baseline** (the papers use "whitened returns" — subtract the mean, divide by the standard deviation, Chapter 5) reduces the noise in the learning signal without biasing it, so training is more stable — it's a variance-reduction trick, not a change of goal. And **ϵ -greedy exploration** means: usually act greedily, but with small probability ϵ take a random action, so the agent keeps discovering alternatives instead of prematurely locking in. The exploration papers sweep ϵ on a decaying schedule (start curious, settle down) and show that *how* you explore — randomize when genuinely uncertain about which action is better, not merely when prediction error is high — decides whether the agent ever becomes competent.

In our papers — value of information, done right and done wrong

Several papers ask when an agent should pay to *probe* (run a costly experiment on its own world). The naive rule — "probe where my current prediction error is high" — is shown to be wrong, because high error can be irreducible noise you can't learn from. The correct quantity is the **value of information**: the *expected reduction in future error* from probing, written $\mathbb{E}[MAE_{\text{after}} - MAE_{\text{now}}]$, or in shrinkage form $\text{error}^2 / (K + 1)$ — big error but many prior

samples (large K) means little left to learn, so low probe value. This is decision theory's classic "value of information" made into a trainable head, and watching the program discover that current-error $\neq$ information-value (and then engineer a signal that tracks the right thing) is one of its most instructive arcs. You now have every piece to read those results: expected value, MAE, the K -samples shrinkage, and the argmax that turns a value estimate into an action.

17.5 What you can now read

MDPs (state, action, transition, reward) and policies π ; value $V(s)$ as discounted expected future reward; the discount γ as a patience dial and γ^l as its decay; greedy action $a^* = \text{argmax score}$; predictive policy closure (a consequence model used as the policy); REINFORCE as reward-driven gradient descent, with baselines/whitened returns as variance reduction and ϵ -greedy as exploration; and value-of-information $\mathbb{E}[\text{MAE_after} - \text{MAE_now}]$ vs the current-error trap.

CHAPTER 18

Causality, Interventions, and Identifiability

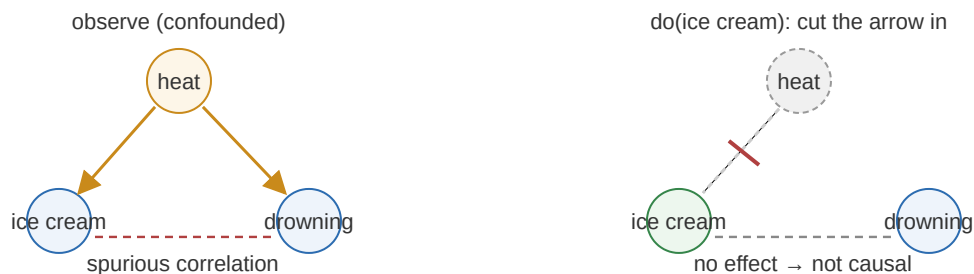
The benchmark half of the program is obsessed with one distinction: does an agent succeed because it tracks the *cause*, or because it latched onto a *correlate* that happens to work? Answering that requires the mathematics of causality — the do-operator, interventions, and identifiability. This chapter builds them, and with it you can read the causally-grounded-agents papers, the null-intervention and gauge-fixing results, and the "probe AUC is not causal use" theorems.

18.1 Correlation is a shadow; causation is the object

Ice-cream sales and drowning deaths rise together — both are caused by summer heat; neither causes the other. This is the oldest lesson in statistics, but the program takes it further than usual: it argues that most claims of machine "understanding" are ice-cream-and-drowning errors, and builds benchmarks specifically to catch them. The tool that separates correlation from causation is the **intervention**.

Definition — observation vs intervention, and the do-operator

$P(Y | X = x)$ — *observing* X to be x — asks "among cases where X happened to be x , how is Y distributed?" It includes all the confounding. $P(Y | do(X = x))$ — *intervening* to set X to x — asks "if I reach in and force X to x , holding nothing else artificially, how is Y distributed?" The **do-operator** means you *made* it happen rather than *found* it happening. Interventions cut the influence of everything upstream of X , so only X 's genuine downstream effects remain.



Intervening severs the confound. Observationally, ice cream and drowning move together via their shared cause, heat. Force ice-cream sales up by decree — $do(\text{ice cream})$ — and drowning doesn't budge: the intervention reveals there was never a causal arrow. The program applies exactly this test to a model's internal features: force one, watch the decision.

18.2 Activation patching: the do-operator, inside a network

You can run interventions *inside* a trained model. **Activation patching** means: reach into the network, overwrite a specific internal feature with a different value (or a corrupted one), and measure how much the output changes. That change is the feature's causal effect on the decision — $do(\text{feature} = \text{value})$ made literal. The long-horizon-bottleneck papers patch a "value prefix" feature and measure the shift in the next-token logit margin (Chapter 12's log-odds); a big shift means the feature genuinely drives the answer.

In our papers — why "a probe can read it" is not "the model uses it"

This is one of the program's sharpest and most-repeated points, and it is pure causality. You can often train a **probe** — a little classifier — to decode some concept from a model's activations with near-perfect accuracy (AUROC $\approx$ 0.999). Tempting conclusion: "the model represents and uses this concept." But decodability is *observational*: the information being present is not the same as the model *acting* on it. The gauge-fixed-transport experiments make the gap vivid — a distractor feature scored AUC 0.999 yet, when patched, had a causal effect $\sim 36\times$ smaller than the real feature. The commitment-surface paper turns this into a theorem: two features can have *identical* probe AUC while their causal patch-effects differ arbitrarily, so **probe accuracy provably cannot lower-bound causal use**. The moral, stated as a benchmark law: *final performance (or decodability) is not evidence that the intended structure is load-bearing* — only an intervention at the commitment surface is. This is why the whole benchmark family insists on structure-specific gates, not just accuracy.

18.3 Identifiability and gauge, revisited with teeth

Chapter 9 introduced gauge symmetry through the self/world split: when the data only reveals a sum, the decomposition into parts is underdetermined — a **gauge freedom**, unfixable by more passive data. This is a special case of an **identifiability** question: *is the quantity I want even determined by the data I can collect?* Sometimes the honest answer is no, and recognizing that converts an apparent "learning failure" into a structural fact — one of the program's most valuable moves, because it stops you from chasing an impossible target with bigger models.

The resolution is always the same shape: **an intervention breaks the gauge**. If the agent can *do nothing on purpose* — the null action, $do(a = \text{null})$ — it observes the world's contribution in isolation, which pins the free constant. The identity the null-intervention paper uses is exactly a difference of interventions:

$$\text{self_}\Delta E = \Delta E_{\text{observed}}(\text{consume}) - \Delta E_{\text{observed}}(\text{null})$$

The world's effect is present in both terms and cancels; what's left is the action's own causal contribution. Reported: 82% reduction in false self-credit.

This is causal inference's core trick — a *contrast between interventions* isolates an effect by differencing out everything common to both. The interventional-contrast papers extend it to supervising on the difference between high-hazard and low-hazard conditions; the online- identifying-interventions papers automate *when* to run such probes. Every one of them is a variation on: to learn a cause, you must sometimes act, not just watch.

Intuition pump — the light switch and the double-blind

To learn whether a switch controls a bulb, you don't correlate switch-positions with bulb-states over a diary of a year (maybe someone else flips it, maybe daylight confounds you). You *flip it yourself* and watch — one deliberate intervention beats a mountain of observation. Medicine formalizes this as the randomized controlled trial: *randomly assign* treatment so nothing upstream correlates with who gets it. The program's null actions, matched controls, and patch experiments are the same instinct pointed inward at a neural network: to know what a part *does*, intervene on it and hold everything else fixed. This is why the papers spend so much effort on controls designed to fail — a control is a pre-planned intervention whose flat result certifies that your signal means what you think.

18.4 What you can now read

Correlation vs causation; observation $P(Y|X)$ vs intervention $P(Y|\text{do}(X))$ and the do-operator; activation patching as the do-operator inside a network; probes and the theorem that decodability (AUC) cannot bound causal use; identifiability and gauge freedom as "the data can't determine this"; gauge-breaking by the null action $\text{self_}\Delta E = \Delta E(\text{consume}) - \Delta E(\text{null})$; interventional contrasts as effect-isolation; and why controls-designed-to-fail are interventions in disguise.

PART VI

The Program's Own Mathematics

You now hold every tool the program uses. This final part assembles them into the ideas the program *invented*: weakness as a theory of generalization, the handful of theorems that tie the whole edifice together, and — the payoff — a guided walk through one real paper from title to conclusion, followed by a glossary you can keep at your elbow.

CHAPTER 19

Weakness, from First Principles

If the program has one central idea, it is **weakness**. It is the through-line from the symbolic benchmarks to the grid cells to the theory papers to the agency work. This chapter builds it slowly, because building it slowly is the point: every apparent complication is one of the simple pieces you already own.

19.1 The question weakness answers

Here is the problem, stripped bare. You have training data. Many different rules fit it perfectly. They *disagree* on new data. Which one do you bet on? This is the problem of **generalization**, and it has no answer from the data alone — the data is, by construction, silent about the cases you haven't seen. You need a *selection principle*: a reason to prefer one fitting rule over another. The history of the field offers one dominant answer — **simplicity** (Occam's razor, MDL, "shortest description wins," Chapter 12). The program proposes a different one — **weakness** — and, crucially, sets up experiments where the two give *opposite* advice, so the winner can be read off.

19.2 Two meanings of weakness, and why they're the same

The word "weakness" is used two ways in the papers, and seeing they coincide is the key insight.

Definition — weakness, extension form (Bennett)

A hypothesis's **extension** is the set of cases it accepts / permits. Its **weakness** is the *size* of that set:

$$w(h) = | \{ x \in X : h \text{ accepts } x \} | \quad (\text{often reported as } \log_2 \text{ of this count})$$

A rule that accepts many possibilities is *weak* — it forbids little, commits to little. A rule that accepts few is *strong*. "The password is 0000" is strong (accepts one world); "the password is four digits" is weak (accepts ten thousand).

Definition — weakness, symmetry form (equivariance count)

A function's weakness is the number of a symmetry group's transformations it is compatible with (Chapter 14):

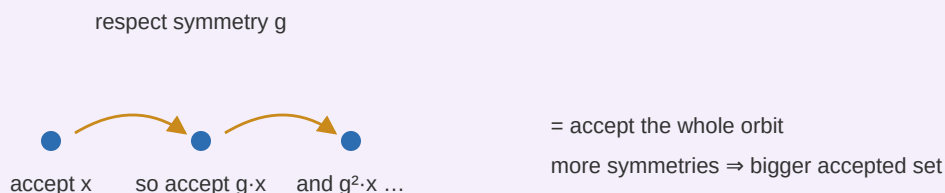
$$W_G(f) = | \{ g \in G : \exists h, \forall x, f(g \cdot x) = h \cdot f(x) \} |$$

A function compatible with many symmetries treats whole orbits alike — it too commits to little beyond "respect these symmetries."

Visual proof — the two forms are one

Why is "accepts a big set" the same as "respects many symmetries"? Because **respecting a symmetry is a way of accepting a set**. If f is compatible with symmetry g , then whenever f accepts x , consistency forces it to accept $g \cdot x$, and $g \cdot (g \cdot x)$, and so on — f must accept the entire *orbit* of x . So each symmetry the function respects *enlarges the set it must accept* by whole orbits. Count the symmetries, or count the accepted set: up to the bookkeeping the concern-weighted-weakness paper works out (it proves the deployment set decomposes into one block per symmetry, so counting compatible symmetries with equal-weight blocks equals counting the accepted set), **they are the same**.

number. The paper's Corollary 3.1 states exactly this: $\operatorname{argmax}_h \operatorname{Weak}_U(h) = \operatorname{argmax}_h |C_G(h)|$ — selecting the largest-extension hypothesis and selecting the most-symmetric one are the same choice.



Symmetry compatibility forces set membership. Each respected symmetry drags in a whole orbit of accepted cases. That is why the count-of-symmetries and the size-of-extension definitions of weakness measure the same commitment-to-little.

19.3 Why weak beats simple (the intuition, then the evidence)

Intuition pump — the over-eager detective

Two detectives explain the same three clues. Detective Simple names a single suspect who fits all three exactly — a tight, short story. Detective Weak says "the culprit is left-handed," which fits the clues but names no one in particular. Now a fourth clue arrives from a case you couldn't see before. Detective Simple's story, having bet everything on fitting the first three, is often just *wrong* about the fourth — it over-committed to the specifics of what it had seen. Detective Weak's looser claim, having only asserted a broad compatible constraint, more often survives. The program's wager is that under distribution shift — when deployment differs from training — the weak-but-compatible hypothesis travels better than the simple-but-brittle one, precisely because it *didn't* spend its credibility fitting the training set's accidents. Weakness is disciplined under-commitment.

That is a story; the program's contribution is turning it into a measured horse-race. The flagship weakness-invariance benchmark constructs tasks (cyclic shifts, dihedral reflections, parity cosets) where a local shortcut and a global symmetric rule both fit the training data perfectly, then asks eleven different selectors to pick the winner and checks which pick generalizes out-of-distribution. The reported result is stark: the weakness selector picks the symmetric, generalizing rule in essentially 100% of trials, while **simplicity, description length, MDL-style priors, compression, held-out validation, training loss, parameter norm, and flatness/ sharpness all pick the shortcut** and generalize badly. And the effect survives the program's control battery — wrong-group weakness (score against the wrong symmetry) and random-label controls fail as they should, and a residualized regression (Chapter 6) shows weakness still predicts OOD accuracy ($\beta \approx +0.37$) after accounting for data augmentation.

19.4 Measuring weakness when you don't know the symmetry

A fair objection: real problems don't hand you the group G . The learned-symmetry-discovery paper answers it. From training data alone, it searches candidate transformations and keeps those under which the data is *self-consistent* — scoring each candidate rotation θ by the average cosine (Chapter 7) between a rotated image and same-class images:

$$s(\theta) = \frac{1}{N} \sum_i \max_{j: \text{same class}} \cos(\text{flatten}(R_\theta x_i), \text{flatten}(x_j))$$

Keep the high-scoring angles as the discovered group, then compute weakness against *that*. The result: weakness scored against the *learned* group retains about 90% of the oracle group's predictive and causal signal (correlation with OOD +0.66 vs oracle +0.74; causal augmentation lift 90.7% of oracle's +51.5 pp). Weakness is not a quantity you can only compute with privileged knowledge; it can be bootstrapped from the data. (The honest caveats are in the paper too: a random-angle "control" wasn't a clean null because random subsets of evenly-spaced angles overlap the true group — the kind of disclosed imperfection Chapter 6 taught you to look for.)

19.5 Concern-weighted weakness: not all cases matter equally

The final generalization closes the loop to agency. Plain weakness counts accepted cases as equals, but a living system cares about some outcomes far more than others. **Concern-weighted weakness** replaces the count with a concern-weighted sum (Chapter 5's expectation idea, and Chapter 11's integral in the continuous case):

$$Weak_{U,\kappa}(h) = \sum_{z \in Z_h \cap U} \kappa(z)$$

Sum the concern κ over the accepted cases in the deployment region U . Set $\kappa \equiv 1$ and you recover plain weakness; let κ vary and important cases count more.

The concern-weighted-weakness paper proves this unifies the whole family: Bennett's extension weakness, the equivariance-count weakness, and this concern-weighted version are all special cases of one weighted-extension selector. And it connects to the geometry of Chapter 9: a finite-capacity code should allocate its resolution proportional to concern-weighted importance, which — through the rate–distortion derivation of Chapter 11 — is exactly the reward-deformation law $\rho^*(x) \propto w(x)^{d/(d+2)}$. The circle closes: **weakness (how much you keep open), concern (what you care about keeping open), and geometry (where you spend resolution to keep it) are three views of one optimization**. That unification, more than any single experiment, is what the program is reaching for.

19.6 What you can now read

Weakness as a selection principle for generalization; the extension form $w(h) = |\text{accepted set}|$ and the symmetry form $W_G(f)$, and the proof they coincide; why weak (under-committed) beats simple (brittle) under distribution shift, and how the benchmark measures it against MDL/validation/norm/sharpness; data-driven symmetry discovery via self-consistency scoring; and concern-weighted weakness $\Sigma\kappa(z)$ as the unifying selector that ties weakness to concern and to the reward-deformation geometry.

CHAPTER 20

The Core Theorems, Assembled

The program has a handful of load-bearing results — a few equations that everything else leans on. You have met them all in pieces; this chapter gathers them in one place, fully decoded, so you can see the architecture whole. Treat it as a reference: when a paper cites "the bridge theorem" or "the Law of the Stack," come here.

20.1 The reward-deformation law

$$\rho^*(x) \propto w(x)^{d/(d+2)}$$

What it says. A finite-capacity code minimizing value-weighted distortion should place its resolution ρ (Chapter 9's $\sqrt{\det g}$) in proportion to concern w , raised to the power $d/(d+2)$, where d is the space's effective dimension. **Where it comes from.** The four-line Lagrangian derivation of Chapter 11 (coding fact + priced budget + zero-derivative + algebra). **Why it matters.** It is the program's one *quantitative, pre-registered, falsifiable* geometric prediction — and it was falsified productively: the measured log-log slope was ≈ 0.30 , not the 0.50 predicted for $d = 2$, implying $d_{\text{eff}} \approx 1$ (the code concentrated its resolution along essentially one direction). **How to read it in a paper.** Any mention of "metric deformation," "effective dimension," " d_{eff} ," or " $\alpha = d/(d+2)$ " is this law.

20.2 The Law of the Stack

$$w(\zeta_{i+1}) \leq 2^{w(\zeta_i)}$$

What it says. The weakness (flexibility, Chapter 19) a layer ζ_{i+1} can have is capped by 2 raised to the weakness of the layer below it — each bit of slack underneath doubles the room available above (Chapter 3's 20-questions logic, stacked). **Why it matters.** It makes "a viable system needs slack in its foundations" a numerical, testable ordering rather than a slogan; the autopoietic-control paper checks whether a model's capacity to repair itself grows with lower-layer slack exactly as the Law predicts. **Companion quantity:** the viability buffer $|B_\pi| = w(\pi) - |\alpha|$ — flexibility minus commitments, i.e. how much shock a policy can absorb before dying.

20.3 The unification of weakness selectors

$$\text{Weak}_{U,k}(h) = \sum_{g \in C_G(h)} \mu_g \quad (\text{and, with equal blocks}) \quad \text{argmax}_h \text{Weak}_U(h) = \text{argmax}_h |C_G(h)|$$

What it says. Concern-weighted extension mass equals a concern-weighted count of compatible symmetries; in the equal-weight case, choosing the largest-extension hypothesis is *identical* to choosing the most-symmetric one. **Why it matters.** It proves the two definitions of weakness (Chapter 19) are one selector seen from two sides, and folds concern into the same framework — the theoretical spine connecting the symbolic benchmarks to the agency work. **How to read it.** $C_G(h)$ is "the symmetries h respects"; μ_g is "how much concern lives in symmetry g 's block"; the argmax equality is the punchline.

20.4 The Bayes-optimality-and-its-limit result

$$\mathbb{E}_g[\operatorname{acc}(f)] = \frac{W_{G_{\text{dep}}}(f)}{|G_{\text{dep}}|} \Rightarrow \operatorname{argmax} W_{G_{\text{dep}}} \text{ is Bayes-optimal (but only when the probe group = the deployment group)}$$

What it says. Expected deployment accuracy equals normalized weakness measured against the *true deployment symmetry group* — so picking maximum weakness is the optimal bet (Chapter 5's Bayes reasoning) *when you score against the right group*. Score against a *superset* group and the ranking can invert (the paper gives an explicit counterexample). **Why it matters.** It says precisely when weakness is trustworthy and when it isn't — weakness's implicit prior must match reality. "Whose group is it?" becomes an experimental variable, not a philosophical aside.

20.5 The gauge-fixed load-bearing bridge

$$\operatorname{Load}_V(h) \geq (\alpha - \delta) \cdot \varepsilon$$

What it says. A distinction is load-bearing (genuinely used, Chapter 18) by at least (initial concern mass α , minus transport loss δ) times the gauge-fixed commitment effect ε . **The three factors, each a chapter of this book:** α is concern mass (Chapter 19's weighted count); δ is how much of it degrades under context change (Chapter 12's distribution distances bound it, via the telescoping "transport bottleneck" $W_n \leq \min_i W_i$); ε is the causal effect of the distinction on the decision *after* an intervention fixes the gauge (Chapter 9 + 18). **Why it matters.** It is the program's formal statement of "availability is not use": mass that survives transport and still moves the decision under intervention is real; anything less is decoration. Validated on a GPU suite where a probe's AUC 0.999 coexisted with a causal effect $\sim 36\times$ smaller than the genuine feature.

20.6 The non-identification proposition

$$\operatorname{AUC}(p \cdot f) = \operatorname{AUC}(p \cdot f') \text{ yet } \operatorname{CE}(f \mid \text{patch}) - \operatorname{CE}(f' \mid \text{patch}) = \delta \text{ (arbitrary)}$$

What it says. Two features can have identical probe accuracy while their causal patch-effects differ by any amount you like. **Why it matters.** It is a clean impossibility result: *decodability cannot lower-bound causal use*, so no amount of probe accuracy proves a model *uses* a feature. This single proposition is the theoretical justification for the entire causally-grounded-agents benchmark family — why they demand intervention gates, not accuracy.

In our papers — how the theorems interlock

Read together, these are not six results but one story in six equations. *Weakness* (§20.3) says: prefer hypotheses that keep the most concern-weighted futures open. *Bayes-optimality* (§20.4) says: that's the right bet exactly when your symmetry prior matches deployment. The *reward-deformation law* (§20.1) says: a finite code physically implements this by warping its geometry to spend resolution where concern is high. The *Law of the Stack* (§20.2) says: it can only do so if lower layers supply enough slack. And the *bridge* (§20.5) plus *non-identification* (§20.6) say: none of this counts as real unless it survives transport and moves the decision under intervention — because a probe reading a feature is not the system using it. Weakness, concern, geometry, viability, and causal use — one optimization, one architecture, six faces. Whether that unification holds up is the program's central open bet, and you are now equipped to help judge it.

20.7 What you can now read

The six load-bearing equations and how each decodes into tools from this book: the reward-deformation law and its falsified exponent; the Law of the Stack and viability buffer; the weakness-selector unification; conditional Bayes-optimality of weakness; the $\text{Load} \geq (\alpha - \delta)\epsilon$ bridge; and the AUC-cannot-bound-causal-use non-identification result — plus how they interlock into a single thesis.

CHAPTER 21

Reading a Real Paper, End to End

Time to cash in everything. We'll read one paper the way you now can — the flagship weakness-invariance paper — from its claim to its final caveat, pausing at each piece of mathematics to name the chapter that decoded it. Then a shorter second pass on an agency paper, to show the same skills transfer. The goal is to leave you certain that no equation in the repository can stop you cold anymore.

21.1 The title and the claim

"Symmetry-Compatible Hypothesis Volume Predicts OOD Generalization." Decode the title first — it usually contains the entire thesis. "Symmetry-compatible hypothesis volume" is **weakness** (Chapter 19): how many symmetries a hypothesis respects = how large a set it accepts. "OOD generalization" is out-of-distribution accuracy — does the rule work on data unlike training. "Predicts" is a claim about **correlation** (Chapter 6). So before reading a word of the body: *this paper claims that how symmetric a learned rule is tells you whether it will work on new, different data*. You could already state the thesis to a colleague.

21.2 The setup — reading the environment

The paper defines tasks like $f(x) = (x + b) \bmod n$. Chapter 14: this is a *rotation of an n -point ring*, the cyclic group $\mathbb{Z}_n$, with a hidden shift b . It also defines $f(x) = (b - x) \bmod n$ — a *reflection*, the dihedral group — and a parity/XOR task on $\mathbb{Z}_2$. The design trick (stated explicitly): each task's training data is fit *equally well* by a narrow "shortcut" and by the true symmetric rule; they diverge only out of distribution. That is the whole experimental idea — engineer a fork where simplicity and weakness point different ways, so the data can adjudicate.

21.3 The central equation — now trivial

$$W_G(f) = |\{ g \in G : \exists h \in G, \forall x, f(g \cdot x) = h \cdot f(x) \}|$$

Walk it with the five-step method from Chapter 1. *Verb*: "=", a definition. *Nouns*: f a learned function, G a symmetry group, g/h its elements. *Loops*: the set-builder filters the symmetries g that pass a test; $|\cdot|$ counts them. *Test*: $f(g \cdot x) = h \cdot f(x)$ is the equivariance commuting square (Chapter 14). *English*: "weakness = how many of the group's symmetries the learned function respects." A quantity that stopped you cold on page one of this book is now a sentence you read at a glance.

21.4 The selectors — a horse race you can score

The paper pits weakness against ten rivals. You know every one: *compression* and *MDL* ($2^{-\text{form_length}}$) are Chapter 12's shortest-code principle; *leave-one-out validation* is held-out accuracy (Chapter 6); *parameter norm* $\|\theta\|$ is Chapter 7's vector length; *sharpness* via the Hutchinson estimator is Chapter 10's curvature-of-the-loss, peeked at with Chapter

5's Rademacher ± 1 random vectors. The result table reports weakness picking the generalizing rule $\sim 100\%$ of the time and every rival picking the shortcut. You can now not only read that table but *appreciate* it — you know what each selector believes and why weakness's win is a real upset, not a definitional trick.

21.5 The statistics — earning the claim

The paper reports Pearson r and Spearman ρ (Chapter 6) between weakness and OOD accuracy, with Fisher- z 95% confidence intervals (a formula-based cousin of Chapter 6's bootstrap; the 1.96 is the 95% z -value from Chapter 5's Gaussian). It runs an augmentation fixed-effect regression and reports a *residual* weakness effect $\beta \approx +0.37$ (Chapter 6's "controlling for" via residuals) — weakness still predicts OOD after data augmentation is accounted for. And it runs the control battery (Chapter 6): *wrong-group* weakness (score against the wrong symmetry — should fail, does) and *random-label* controls. Every statistical move in the paper is one this book built.

21.6 The caveats — reading like a scientist

Mature reading means weighing the limits, and the paper states them: neural sweeps at 256–4096 models are well-powered, but the language appendix is $n = 24$ concepts (underpowered — Chapter 6's $1/\sqrt{n}$ bites); results are on synthetic and small-vision tasks; a single layer per model is probed. None of this sinks the claim, but all of it bounds it — and Chapter 6 taught you that a paper honest about its bounds is doing science right, not hedging. You should finish this paper able to say: "The core symbolic result is strong and well-controlled; the neural extension is solid; the language extension is suggestive but underpowered; and the whole thing lives or dies on whether synthetic symmetry tasks tell us about real deployment — which the later real-model papers then go test."

21.7 Second pass, faster — an agency paper

Now the planning-from-concern paper, at speed, to prove the skills generalize. *Claim (title)*: a greedy planner on a self-organized consequence model achieves homeostatic competence with no action labels. *Setup*: the MDP (Chapter 17) with the homeostat $E \leftarrow \text{clip}(E - 0.04 + \text{reward}, 0, 1)$ (Chapter 16). *Central equation*: $a^* = \text{argmax}_a \Delta E_{\text{head}}(z, E, a) — \text{argmax}$ (Chapter 1) over a learned energy-change predictor: predictive policy closure (Chapter 17). *Training*: MSE (Chapter 12) on observed ΔE via gradient descent (Chapter 10). *Evidence*: reward_gap +1.84 — the centered-cosine cluster gap (Chapter 7) — and return 50/50, against lower-bound (random encoder) and upper-bound (supervised) controls and across 3 seeds (Chapter 6). *Caveat*: 3 seeds, synthetic environment. You read a paper from a completely different corner of the program using the same toolkit, in one paragraph. That transfer — same tools, any paper — is the fluency this book was for.

Intuition pump — you are now a reader, not a decoder

The difference between decoding and reading: a decoder translates each symbol and reassembles the meaning laboriously; a reader takes in the sentence and immediately asks the next question — "is that control the right one? is n big enough? does this follow from the last section?" You crossed that line somewhere in this book. From here, getting better is not about learning more math; it is about reading more papers, each one faster than the last, until the notation is as transparent as prose. The repository is now open to you.

21.8 A checklist for any paper in the repo

Step	Ask	Chapter
1. Title	What is the one-sentence claim hidden in the title?	—
2. Setup	What are the states, actions, inputs, symmetry group? Frozen slices?	2, 14, 16, 17
3. Central equation	Five-step decode: verb, nouns, loops, English, toy case.	1
4. Defined quantities	Is this "score" secretly a filtered count, a cosine, an MAE, a weakness?	4, 7, 12, 19
5. Method	Gates? Seeds? Which controls, and do they fail as they should? Oracle ceiling?	6, 18
6. Effect sizes	Lift, ratio, % reduction, CI lower bound above zero?	6
7. Caveats	Power, external validity, what the authors decline to claim.	6

21.9 What you can now read

Any paper in the repository — by decoding its title into a thesis, its setup into a known structure, its central equation with the five-step method, its selectors and statistics with Part I and Chapter 6, and its caveats with a scientist's eye. The checklist above is your permanent tool.

CHAPTER 22

Glossary and Symbol Decoder Ring

Keep this chapter open while you read. It has two parts: a **symbol decoder** (what each mark means, pronounced aloud) and a **concept glossary** (one-line definitions, each pointing to the chapter that builds it).

22.1 Symbol decoder ring

Symbol	Say	Means
$:= \triangleq \equiv$	"is defined as"	a naming ceremony, not a claim (Ch 1)
$\in \notin \subseteq$	"in / not in / subset"	set membership and containment (Ch 1)
$\{x \in X : P(x)\}$	"the x such that P"	set-builder = a filter (Ch 1)
$ A $	"size of A"	count of a set — the core of weakness (Ch 4, 19)
$\Sigma \Pi$	"sum / product over"	accumulating loops (Ch 1)
$\int f dx$	"integral of f"	continuous sum / area under curve (Ch 11)
$\mathbb{E}[X]$	"expected value"	long-run / probability-weighted average (Ch 5)
$\operatorname{argmax}_a f$	"arg-max"	which a wins, not the winning value (Ch 1)
$\forall \exists$	"for all / there exists"	logic quantifiers (Ch 1)
$\hat{x} \bar{x} x^* x'$	"hat / bar / star / prime"	estimate / average / optimal / a related one (Ch 1)
$x \sim p$	"drawn from p"	random sample from distribution p (Ch 5)
$P(A B)$	"A given B"	conditional probability = zoom into B (Ch 5)
$\operatorname{do}(X = x)$	"do X"	intervene / force X, vs merely observe (Ch 18)
$\mathbf{1}[\cdot]$	"indicator"	1 if true, 0 if false (Ch 2)
$\ v\ $	"norm of v"	length / Euclidean distance (Ch 7)
$u \cdot v$	"u dot v"	$= \ u\ \ v\ \cos \theta$; alignment of directions (Ch 7)
$\cos(u, v)$	"cosine similarity"	$u \cdot v / (\ u\ \ v\) \in [-1, 1]$ (Ch 7)
∇L	"grad L"	uphill direction of the loss; $-\nabla L$ is downhill (Ch 10)
$f'(x), \partial f / \partial x$	"derivative"	how output moves when you nudge input (Ch 10)
$\theta \leftarrow \theta - \eta \nabla L$	—	the universal training step; η = learning rate (Ch 10)
$g \cdot x$	"g acts on x"	apply symmetry g to x (Ch 14)
$f(g \cdot x) = h \cdot f(x)$	—	equivariance / compatibility square (Ch 14)
$\mathbb{Z}_n, D_n, S_n$	"cyclic / dihedral / symmetric"	the small symmetry groups (Ch 14)
$J, J^T J$	"Jacobian, pullback metric"	local stretch matrix; induced ruler (Ch 8, 9)
γ	"gamma"	discount / patience in value (Ch 17); also decay rate
κ	"kappa"	concern weighting $\kappa : U \rightarrow \mathbb{R}_{\geq 0}$ (Ch 19)
$\rho, \rho^*(x)$	"rho"	resolution density / optimal allocation (Ch 9, 11)
λ	"lambda"	Lagrange price / penalty strength / eigenvalue (Ch 8, 10, 11)

(b_0, b_1, b_2)	"Betti numbers"	counts of pieces, loops, voids (Ch 15)
-------------------	-----------------	--

22.2 Concept glossary

Term	One-line meaning	Ch
Ablation	subtract a direction from a latent and see if behavior breaks — causal surgery	7, 18
Allostasis	acting in advance to stay viable (forward-looking homeostasis)	16
Attractor / basin	a state that pulls nearby trajectories in; the set that funnels to it	16
AUC (this repo)	often area under an error-vs-time curve (lower=better) — not always AUROC	6
Autopoiesis	a system that actively maintains its own organization; repairs after perturbation	16
Bootstrap CI	error bars by resampling your own data thousands of times	6
Concern / valence	a nonnegative weighting κ of how much outcomes matter to the system	17, 19
Cluster gap (reward_gap)	same-class minus different-class average cosine — how organized the geometry is	7
Commitment surface	where a distinction actually changes the decision; where "use" is tested	18, 20
Cross-entropy	surprise you pay for predicting with the wrong distribution; the classifier loss	12
Effective dimension (d_eff)	how many directions a cloud/code really uses (entropy of the spectrum)	8, 9
Equivariance / invariance	output transforms with / doesn't change under an input symmetry	14
Gauge symmetry	descriptive freedom that changes no observation; unfixable without intervention	9, 18
Gate (pre-registered)	a numeric pass/fail threshold frozen before the experiment — a unit test for a claim	6
Geodesic	shortest path in a space with a varying ruler (metric)	9
Gradient descent	roll downhill on the loss: $\theta \leftarrow \theta - \eta \nabla L$; how every model here trains	10
Homeostat	the $E \leftarrow \text{clip}(E - \text{decay} + \text{reward}, 0, 1)$ survival dynamics of the agency papers	16
Identifiability	whether the data can even determine the quantity you want	18
Information gain	$\log_2$ of how many outcomes a probe distinguishes; bits learned	12
Irrep	an indivisible symmetry pattern; for shifts, a Fourier frequency mode	13, 14
Jacobian / pullback metric	local stretch of an encoder; the ruler $J^T J$ it induces	8, 9
KL divergence	extra bits for using q when truth is p ; a directed distance between distributions	12
Law of the Stack	$w(c_{i+1}) \leq 2^{w(c_i)}$: lower-layer slack caps upper-layer flexibility	16, 20

Load-bearing	a structure that causally drives the decision — not merely decodable	18, 20
MAE / MSE	mean absolute / squared error — the universal metrics and losses	6, 10
Matched-random control	random behavior given the same budget; the bar a learned method must clear	6
MDL	minimum description length: shortest code wins — the rival weakness beats	12, 19
MDP / policy	states-actions-transitions-reward; a rule π for choosing actions	17
Null action / intervention	doing nothing on purpose to observe the world alone and break a gauge	18
Oracle (ceiling)	a condition with cheat access to truth, bounding what's achievable	6
Orbit	everything reachable from x by a symmetry group — a category	14
Patching (activation)	overwrite an internal feature and measure the output change: <code>do()</code> inside a net	18
PCA / spectrum	the natural axes and spreads of a data cloud, from covariance eigenvalues	8
Persistent homology	grow balls around points, count holes that persist — Betti numbers of a cloud	15
Predictive policy closure	a learned consequence model used directly as the policy via <code>argmax</code>	17
Probe	a small classifier decoding a concept from activations — availability, not use	18
Rate–distortion	the bit-budget vs error trade; source of the reward-deformation law	11, 12
Reward-deformation law	$\rho^*(x) \propto w(x)^{d/(d+2)}$: spend resolution by concern; exponent falsified to ~ 0.30	9, 11, 20
Spearman ρ / Pearson r	rank / linear correlation; negative ρ = "anti-calibrated"	6
Torus	product of two circles, Betti (1,2,1); fingerprint of discovered 2-D symmetry	15
Value of information	$\mathbb{E}[\text{error reduction}]$ from probing — the right rule, vs the current-error trap	17
Viability buffer	$ B_\pi = w(\pi) - \alpha $: flexibility minus commitments; shock absorption	16
Weakness	size of a hypothesis's accepted set = number of symmetries it respects; under-commitment	19
z-score	$(x - \text{mean})/\text{std}$: how many wobbles from average; puts different rulers on one scale	6

A closing word

You began this book able to read 10th-grade algebra. You can now decode the central equation of a neural-network generalization paper, follow a Lagrangian derivation to a falsifiable exponent, read a causal-intervention result, and tell a load-bearing structure from a decorative one — using the repository's own papers as your worked examples. The mathematics was never the barrier; the *notation* was, and notation is just frozen sentences. You have learned to thaw them. Every paper in the program is now a door you can open. Go read them — and, armed with the questions this book taught you to ask, help decide which of its bold bets are right.